Install Redis on Windows 10

Author: Sri Adilakshmi M

Table of Contents

1. **Overview:** 3
2. **Install Standalone Redis Server:** 4
   - 2.1. **Install Redis from Microsoft Archive:** 4
   - 2.2. **Verify Redis Installation:** 11
3. **Redis Command Line:** 11
4. **Connect Redis From Python Line:** 12
   - 4.1. **Verify Python:** 12
   - 4.2. **Install Redis Python Library:** 13
   - 4.3. **Connect and Manage Redis:** 14
   - 4.4. **Run Python Script:** 15
   - 4.5. **Async Support:** 19
5. **Deploy Redis Replication:** 22
   - 5.1. **Prepare Configuration:** 22
   - 5.2. **Secure Replication:** 25
   - 5.3. **Start Instances:** 28
   - 5.4. **Verify Connection:** 30
   - 5.5. **Test Data Sync:** 31
6. **Deploy Redis Sentinel:** 32
   - 6.1. **Configure Sentinel:** 32
   - 6.2. **Launch Sentinel Instances:** 36
   - 6.3. **Verify Setup:** 38
   - 6.4. **Test Fail Over:** 40
   - 6.5. **Verify New Hierarchy:** 41
   - 6.6. **Reunion Failed Instance:** 43
   - 6.7. **Connect Sentinel from Python:** 44

This document outlines the steps needed to install Redis server from **Microsoft Archive** and configure 2 deployment modes – **Redis Replication** and **Redis Sentinel** – on Windows Operating system and work with Redis database by executing queries and access through Python API.

1. Overview:

Redis stands for **RE**mote **DI**rectory **S**erver is an open-source (licensed under BSD), in-memory data structure store used for multiple purposes – as a database, cache, message broker, and streaming engine. It is well-known for its exceptional speed and performance since it stores all data in computer's main memory (RAM) rather than on disk which results in sub-millisecond response times making it roughly **1,000 times faster than SSDs** and **100,000 times faster than traditional hard drives**. This extreme speed enables millions of requests per second, making it a popular choice for caching, session management, real-time analytics, gaming, leaderboards, geospatial and messaging systems across sectors ranging from FinTech to Healthcare.

Redis is a NoSQL (Not only SQL) database that stores data as key-value pairs. It is written in ANSI C language and operates natively on POSIX compliant systems like Linux, BSD, and macOS without needing external dependencies. It provides extensive library support for many languages like C, C++, C#, Java, JavaScript, Node.js, Python, PHP, Ruby and R.

The Redis project was originally developed by **Salvatore Sanfilippo** in 2009 to optimize a real-time analyzer application. Since then, Redis has been adopted by many major companies like **Twitter, GitHub, Instagram, Snapchat**, etc. The company behind the project, **Redis Labs** has officially rebranded to **Redis** in 2021.

Many cloud service providers offer managed Redis solutions, such as Amazon ElastiCache *(offered by **Amazon Web Services**)*, Memorystore *(offered by **Google Cloud Platform**)* and Azure Cache for Redis *(offered by **Microsoft Cloud**)* with premium features like automated scaling and backups. The Redis company itself offers a managed service called Redis Cloud with advanced capabilities such as automated sharding, tunable durability, and specialized modules for JSON, search, and graph processing.

Redis can be deployed with various architectures:

1. **Redis Standalone** – Standalone is the most straightforward and simple deployment consisting of a single Redis instance. This allows users to setup small instances which helps to speed up the application performance. This is primary used for development, testing and small-scale caching of application.

2. **Redis Replication –** Redis Replication deployment contains one or more secondary instances along with the main instance, following the master – slave pattern. In Redis replication, the primary (master) instance handles all write requests and sends the copy of those commands

to one or more secondary (replica) instances for data synchronization. This ensures the replica Redis instances are exact copies of master instance, providing scale performance and failover when the main instance is unavailable, enabling high availability (HA) of the system.

3. **Redis Sentinel –** Redis Sentinel is the distributed monitoring solution that handles the automatic failover of Redis masters and service discovery very much like **Zookeeper** and **Consul** in other systems. It works by running a set of separate processes called **Sentinels** that monitor the Redis master and replica nodes to ensure the system remains available without manual intervention. These Sentinels should be placed on separate physical or virtual machines to ensure they fail independently.

4. **Redis Cluster** – Redis Cluster is the distributed implementation designed for horizontal scaling and high availability by automatically partitioning (sharding) data across multiple nodes. It comprises of an interconnected group of master and replica instances where masters handle read/write requests and data partitioning and replicas maintain exact copy of their master's data and provide fault tolerance via automatic failover, ensuring high availability.

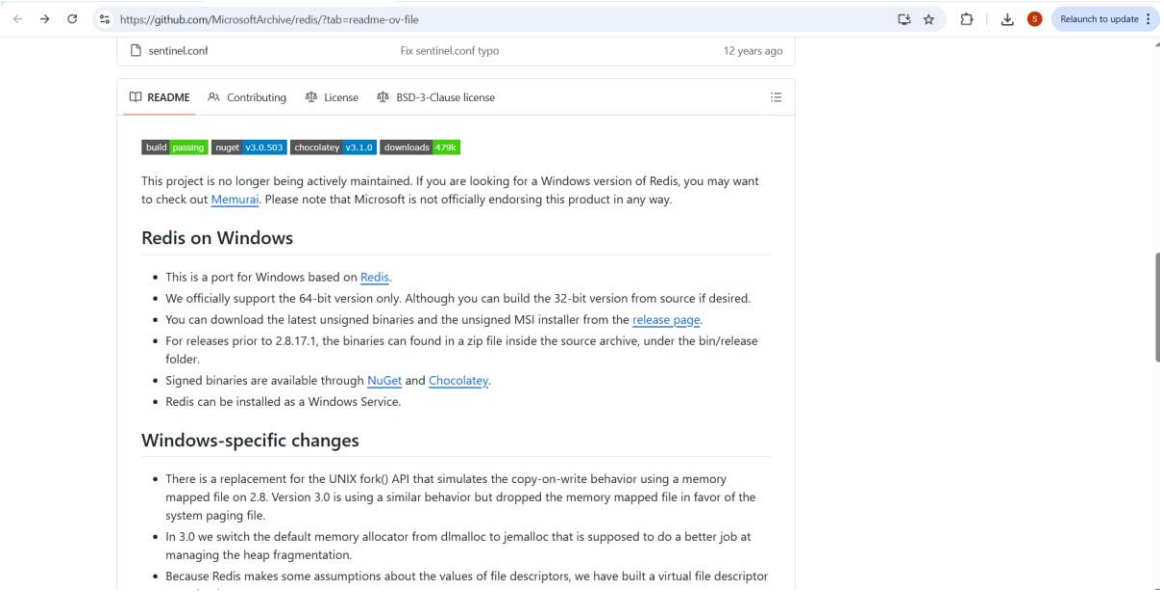
2. Install Standalone Redis Server:

Redis is designed to run POSIX systems like **Linux** and **macOS**. While Windows is not officially supported, the recommended installation of Redis can be done using **WSL2** (Windows Subsystem for Linux) or **Docker** to run the standard Linux version of Redis directly on Windows.

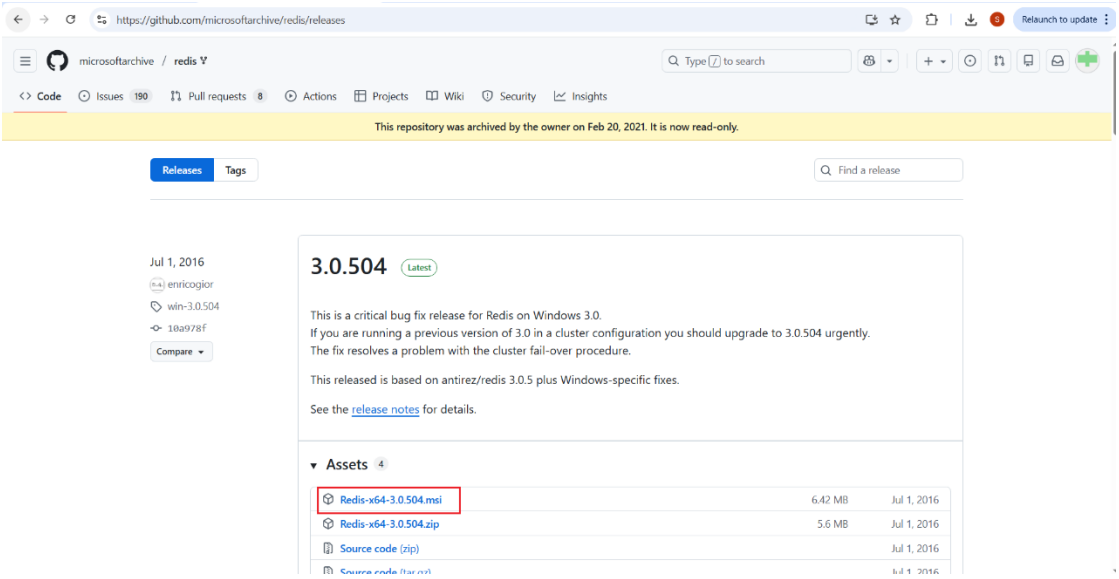
If WSL or Docker cannot be used, Memurai is recommended, offering a standard native port for Windows and installs Redis as a standalone service. However, the older versions of Redis such as Redis 3.x can be installed from **Microsoft Archive**.

2.1. Install Redis from Microsoft Archive:

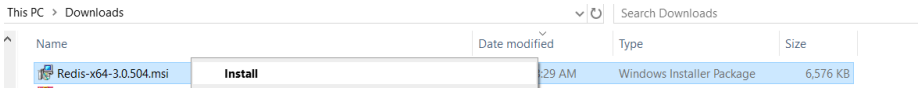
Go to Microsoft Archived Redis Github Repository page which provides an archived version of Redis for Windows that was supported by Microsoft earlier (now, being provided by Memurai). The latest archived version available is **3.0.504**.



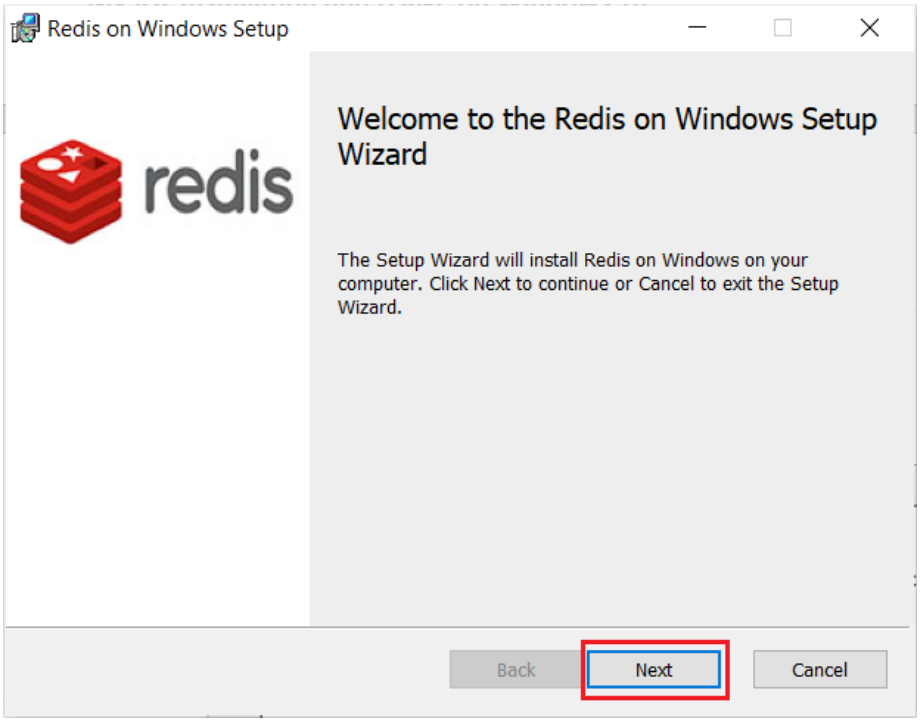
Go to [Microsoft Archived Redis Releases](https://github.com/microsoftarchive/redis/releases) page and click on `Redis-x64-3.0.504.msi` to install the latest unsigned MSI installer which gets downloaded into your **Downloads** folder.



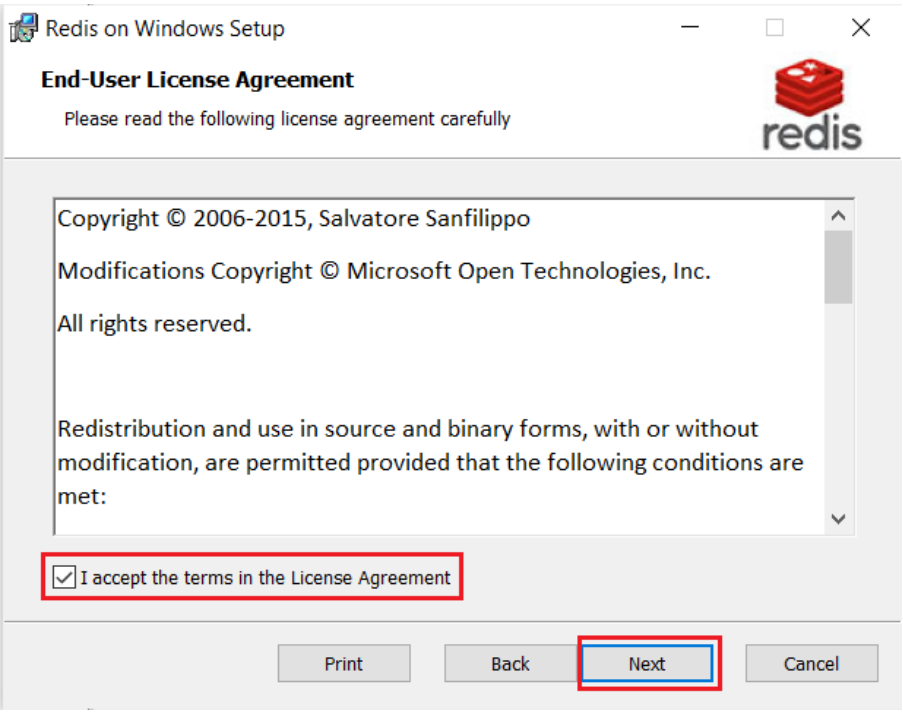
After the file is downloaded, go to your **Downloads** folder and right click on `Redis-x64-3.0.504.msi` and select **Install** option.



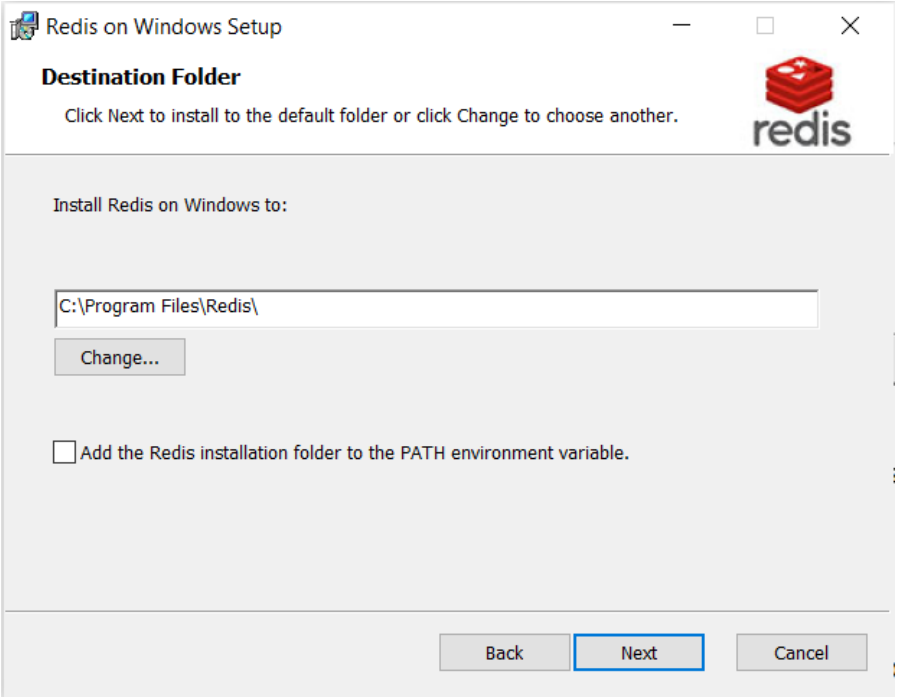
It opens up **Redis on Windows Setup** wizard. Click **Next** to continue.



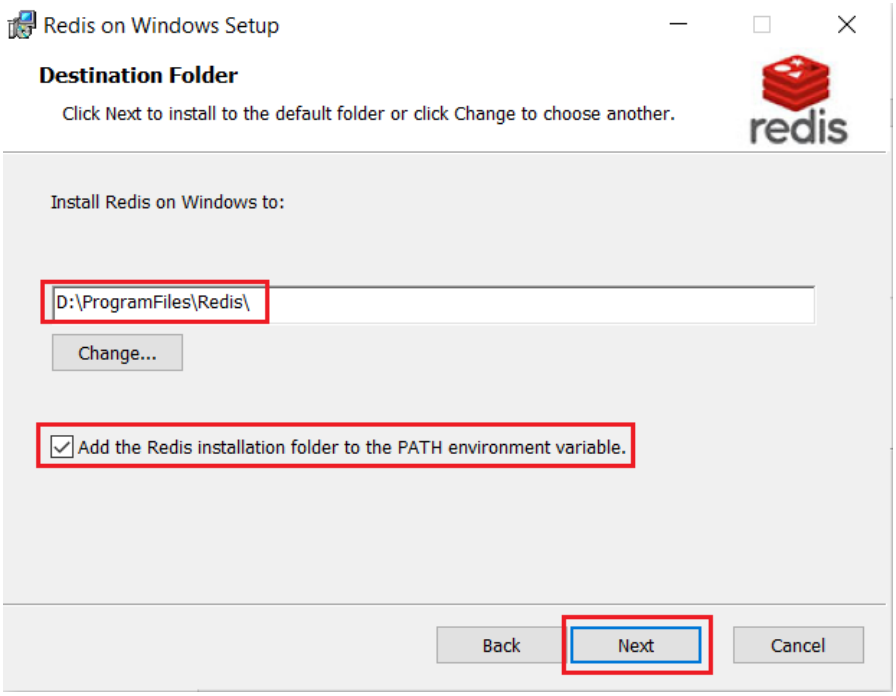
Then, select **I accept the terms in the License Agreement** checkbox and click **Next** to continue.



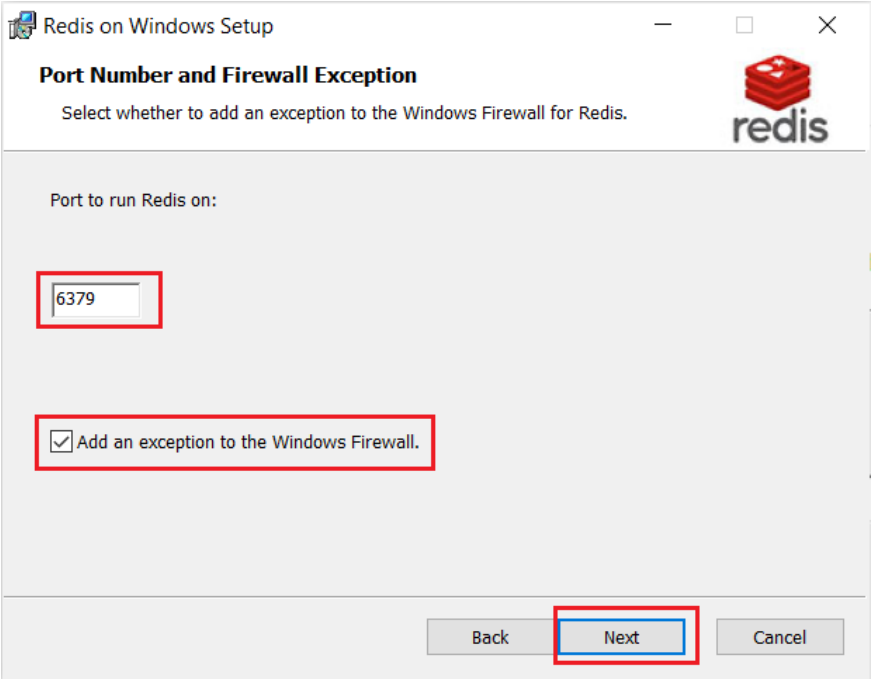
It displays the default installation folder as `C:\Program Files\Redis`.



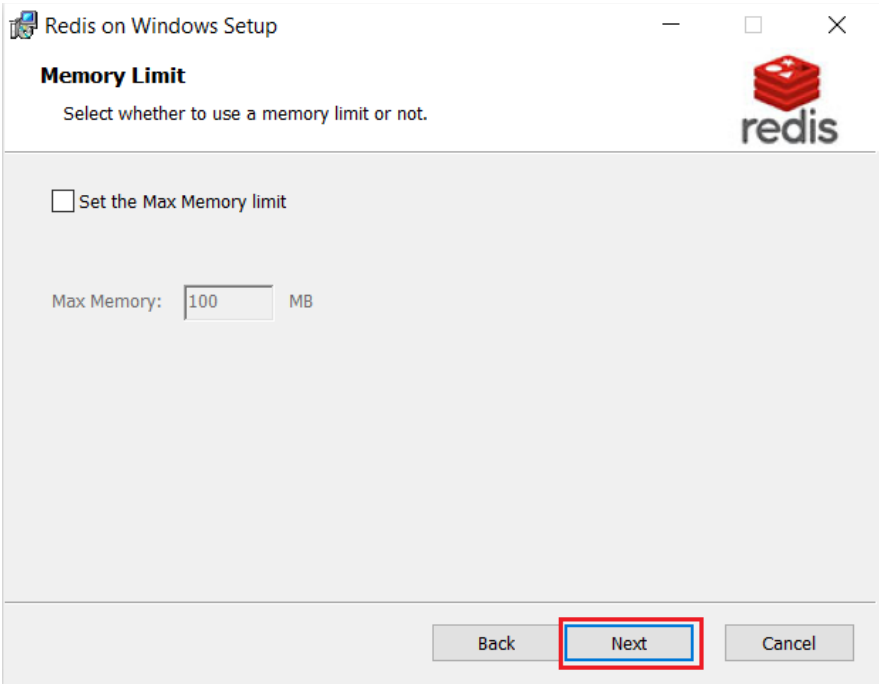
Provide the installation path if needed *(here, I am changing the installation folder to `D:\ProgramFiles\Redis`)* and select **Add the Redis installation folder to the PATH environment variable** check box. Click **Next** to continue.



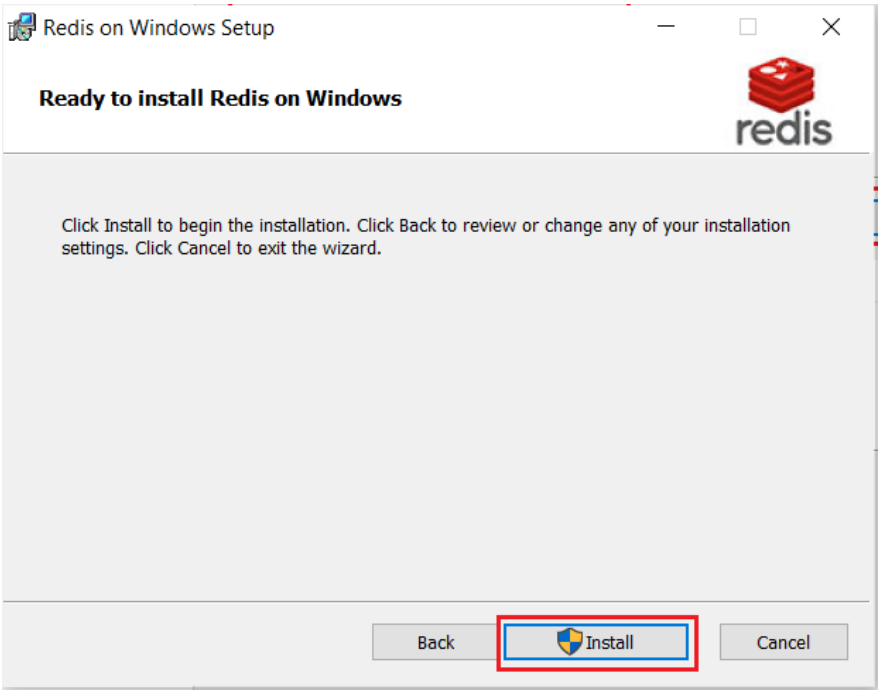
Then, it displays the default port `6379` to run Redis on. Select **Add an exception to the Windows Firewall** check box. Click **Next** to continue.



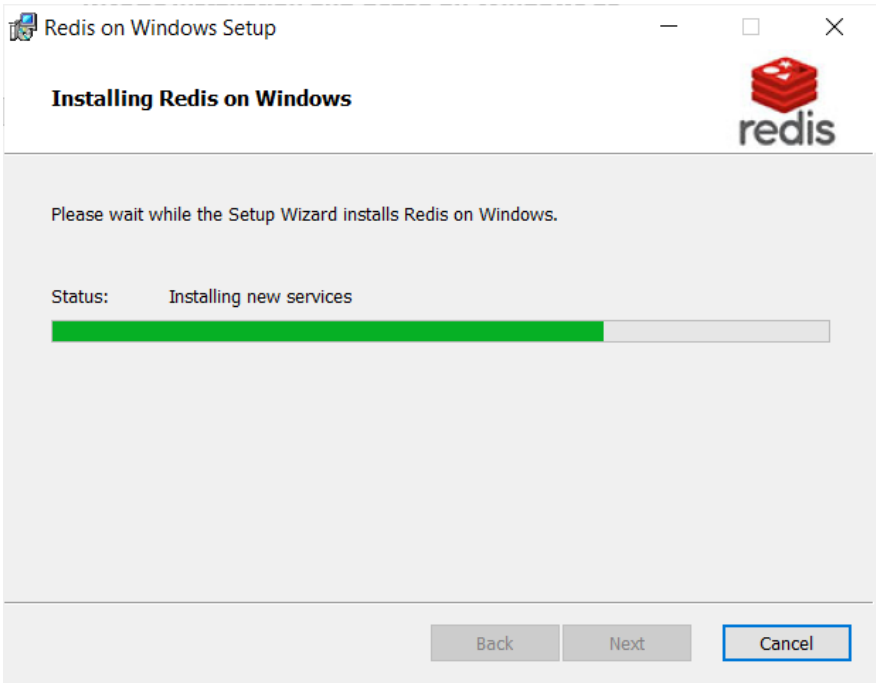
Then, set the maximum memory limit if needed. For now, leave it as it is and click **Next** to continue.



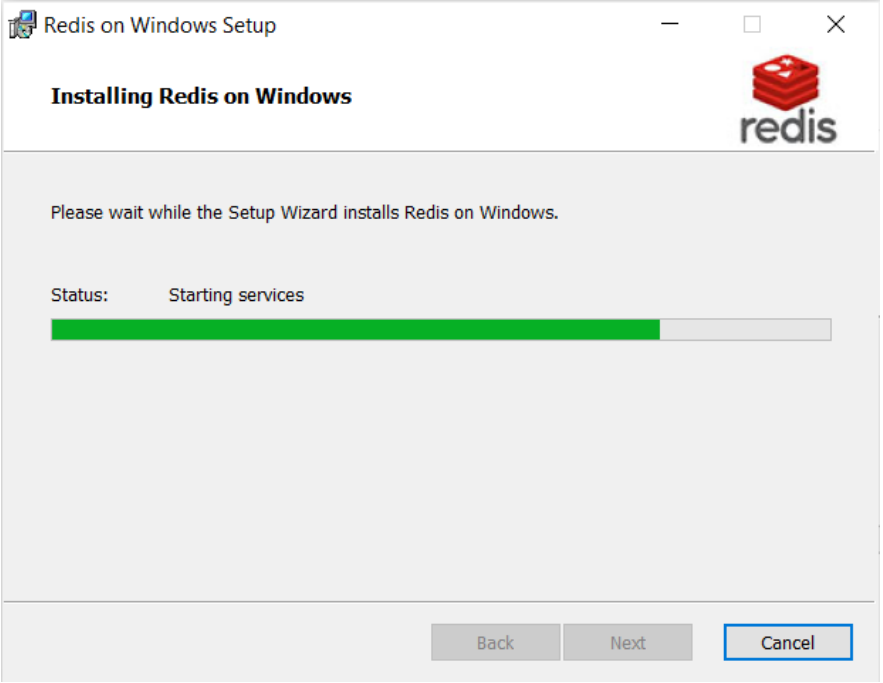
Click on **Install** to start with the installation.



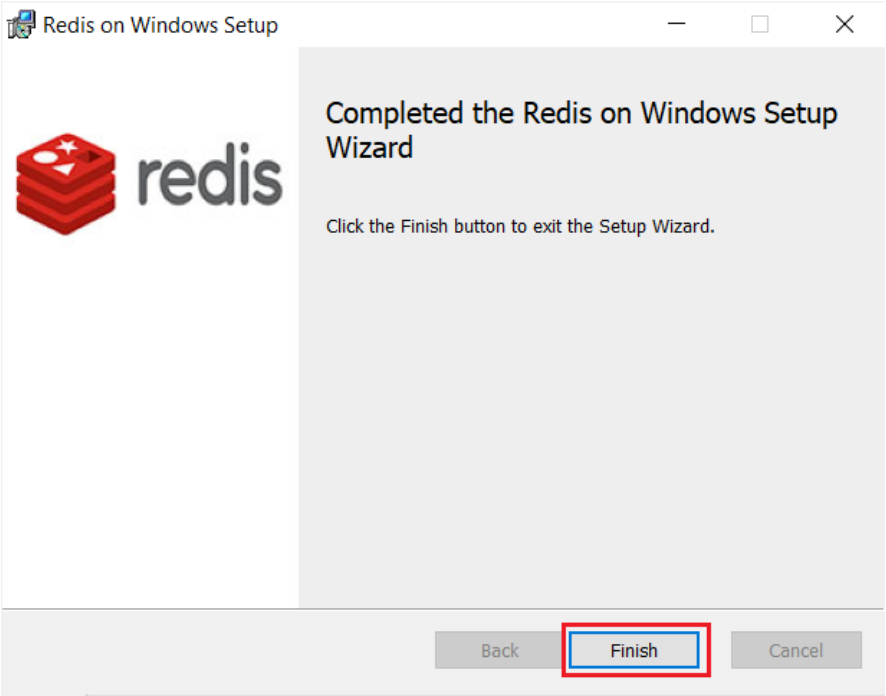
The setup wizard installs the new services for Redis.



Then starts all necessary services for Redis.



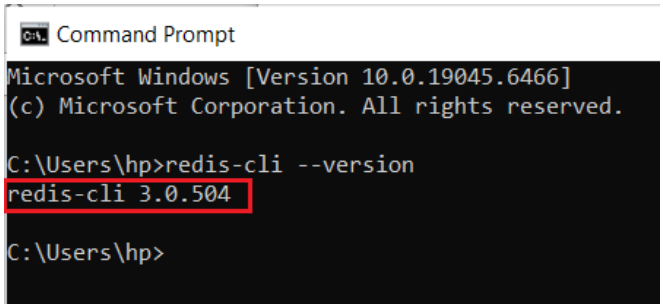
In few seconds, the installation would complete. Click **Finish** to close the wizard.



2.2. Verify Redis Installation:

Open new **Command Prompt** and run the following command to get the installed Redis version:

```
redis-cli --version
```



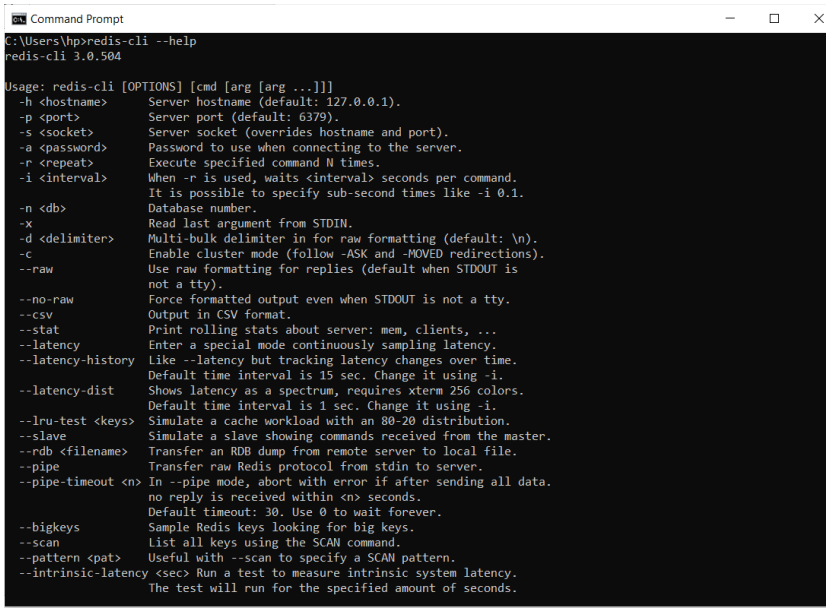
Here, it displays the `redis-cli 3.0.504` version that was installed on the Windows system.

3. Redis Command Line:

Redis provides a client tool called `redis-cli` which is a command line interface to execute commands against Redis database. It supports various options which can be specified in the command line.

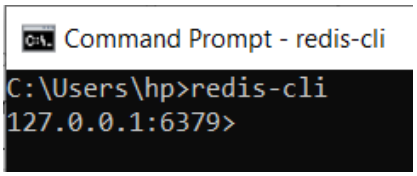
Use the following command to get the list of options supported by `redis-cli`:

```
redis-cli --help
```



Run the following command to connect to Redis server running locally on `6379` port:

```
redis-cli
```

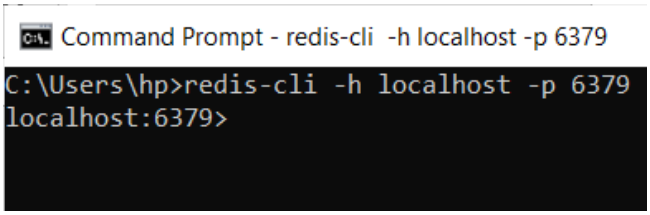


To connect to a Redis instance running on a different host, use `-h` and `-p` along with `--user` and `-a` options as needed:

```
redis-cli -h <host> -p <port> --user <username> -a <password>
```

You can connect to `localhost` instance running on `6379` port as below:

```
redis-cli -h localhost -p 6379
```



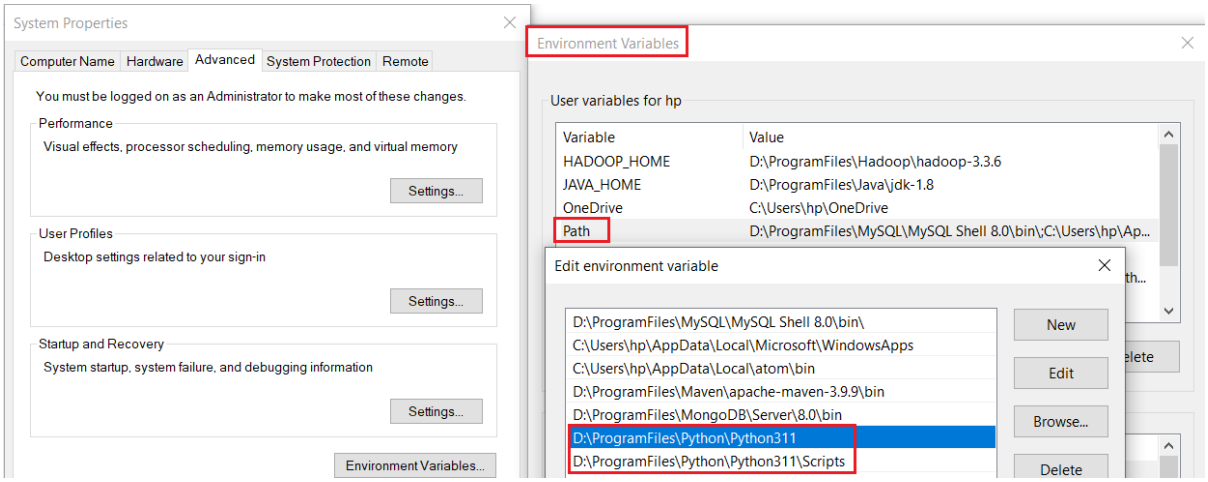
`redis-cli` provides many built-in commands to perform tasks like connecting to a Redis instance, sending command to Redis server, executing a script file, etc. These commands can be executed either by passing as parameters directly to `redis-cli` or using its interactive shell. Detailed command references can be found here.

4. Connect Redis From Python Line:

Redis offers the official Python client redis-py library which provides a comprehensive API to connect to Redis server and execute all Redis commands as Python methods.

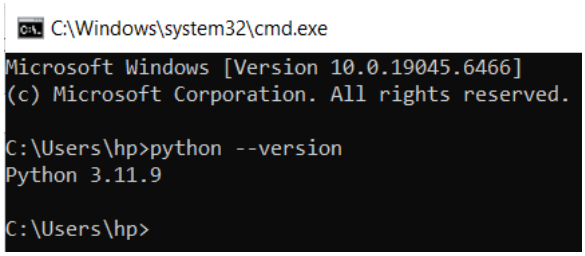
4.1. Verify Python:

If Python is not already installed in your system, install it from the official Python Downloads website and set `Python` install path and `Python\Scripts` path in `PATH` environment variable as below to access Python CLI.



Open a new **Command Prompt** and verify the installed Python version using the below command:

```
python --version
```

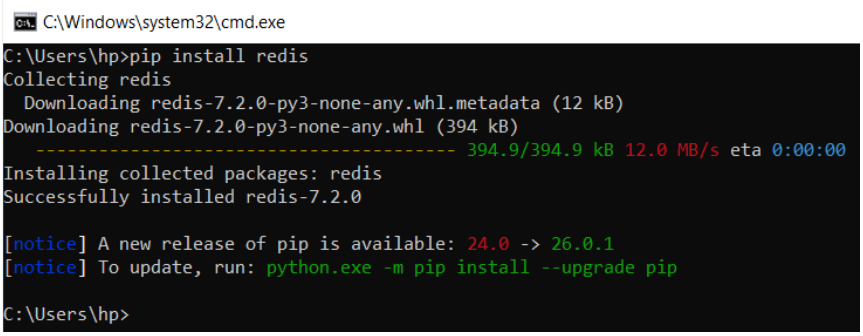


In my system, **Python 3.11.9** version is installed.

4.2. Install Redis Python Library:

Install `redis-py` using the following command:

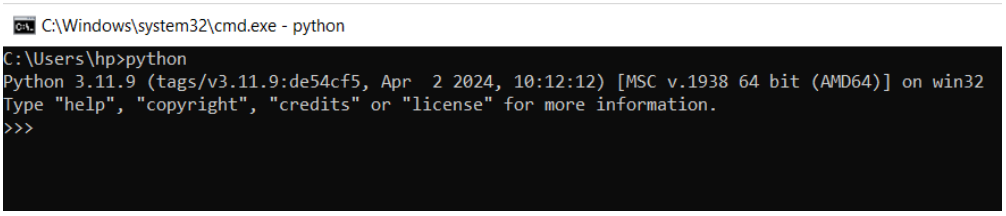
```
pip install redis
```



4.3. Connect and Manage Redis:

Run the following command to launch Python CLI for executing Python code:

```
python
```



In Python CLI, run the following code line by line:

```
import redis
rclient = redis.Redis(host='localhost', port=6379, db=0, decode_responses=True)
print(rclient.ping())
rclient.set('name','John Doe')
rclient.get('name')
rclient.setex('phone',600,'453-3216570')
exit()
```

The above code performs the following:

- Import `redis` library.
- Create a Redis client using `Redis()` method of `redis` library to connect to `localhost` Redis instance running on default port `6379`. This method accepts various arguments such as `host`, `port`, `username`, `password`, `db`, `decode_responses`, etc. *(Since Redis client by default returns the output in bytes, always set `decode_responses=True` to convert Redis byte responses to Python strings automatically).*

  To create a Redis client using a connection string, you can use `from_url()` method of `redis` library as below:

  ```
  rclient = redis.from_url('redis://localhost:6379/0')
  ```

  To connect to Redis with a secure password, use the connection string syntax below:

  ```
  rclient = redis.from_url('redis://:password@localhost:6379/0')
  ```

- Test connection using `ping()` method of Redis client which returns `True` if connection is successful or throws error in case of any issue.

- Set a string key using `set()` method of Redis client by passing key and value as arguments.
- Get a string key using `get()` method of Redis client.
- Create a string key with timeout using `setex()` method of Redis client.
- Exit Python CLI.

Command Prompt

```
C:\Users\hp>python
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import redis
>>> rclient = redis.Redis(host='localhost', port=6379, db=0, decode_responses=True)
>>> print(rclient.ping())
True
>>> rclient.set('name','John Doe')
True
>>> rclient.get('name')
'John Doe'
>>> rclient.setex('phone',600,'453-3216570')
True
>>> exit()

C:\Users\hp>
```

4.4. Run Python Script:

Now, prepare a comprehensive Python script that populates a Redis instance with various data types to demonstrate how the Redis database works.

Create a file named `redis-commands.py` in local system *(in my case, I am creating file under `D:\Learning\Big Data\Redis`)*.

Edit `redis-commands.py` and add the following code:

```
import redis

# Connect to local Redis instance
rclient = redis.Redis(host='localhost', port=6379, db=0,
decode_responses=True)

# Strings (Basic key-value)
print("\n--- Strings ---")
rclient.set('user:name', 'Alice') # Set a key (equivalent to Redis SET
command)
rclient.set('user:email', 'alice@example.com', ex=60) # Set with 60s expiry
rclient.mset({"user:website": "example.com", "user:status": "active"}) #
Multiple set (equivalent to Redis MSET command)
rclient.incr('counter1') # Increment by 1 (equivalent to Redis INCR command)
rclient.incrby('counter1', 5) # Increment by 5 (equivalent to Redis INCRBY
command)
```

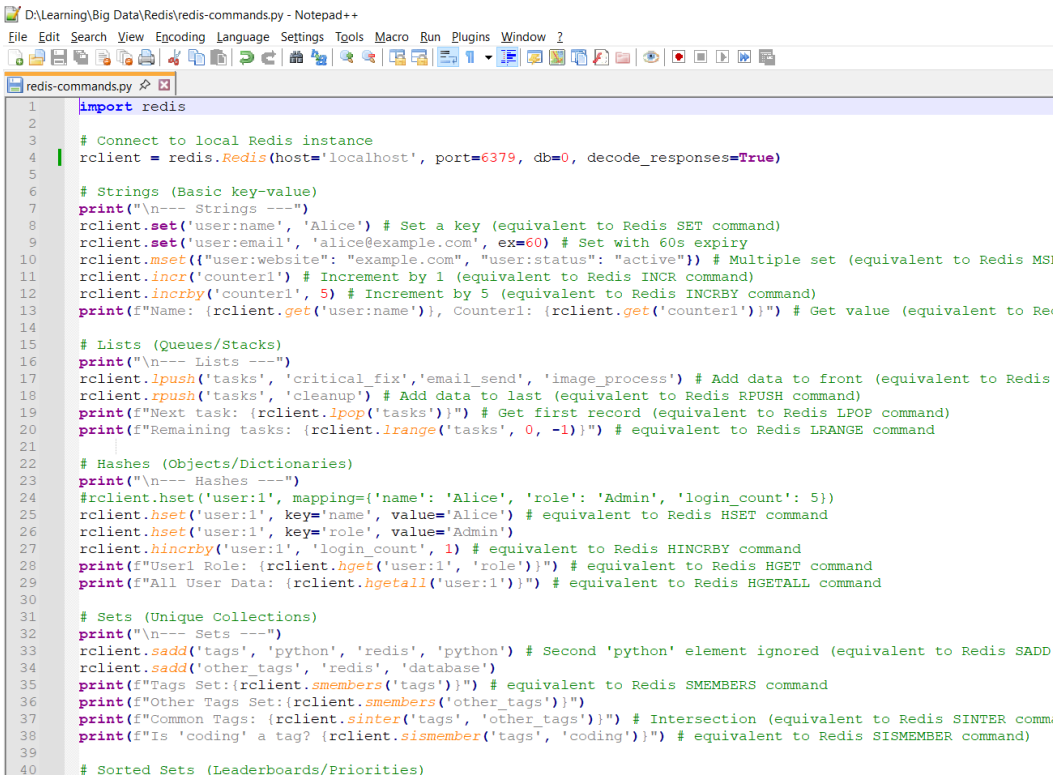
```
print(f"Name: {rclient.get('user:name')}, Counter1:
{rclient.get('counter1')}") # Get value (equivalent to Redis GET command)

# Lists (Queues/Stacks)
print("\n--- Lists ---")
rclient.lpush('tasks', 'critical_fix','email_send', 'image_process') # Add
data to front (equivalent to Redis LPUSH command)
rclient.rpush('tasks', 'cleanup') # Add data to last (equivalent to Redis
RPUSH command)
print(f"Next task: {rclient.lpop('tasks')}") # Get first record (equivalent
to Redis LPOP command)
print(f"Remaining tasks: {rclient.lrange('tasks', 0, -1)}") # equivalent to
Redis LRANGE command

# Hashes (Objects/Dictionaries)
print("\n--- Hashes ---")
#rclient.hset('user:1', mapping={'name': 'Alice', 'role': 'Admin',
'login_count': 5})
rclient.hset('user:1', key='name', value='Alice') # equivalent to Redis HSET
command
rclient.hset('user:1', key='role', value='Admin')
rclient.hincrby('user:1', 'login_count', 1) # equivalent to Redis HINCRBY
command
print(f"User1 Role: {rclient.hget('user:1', 'role')}") # equivalent to Redis
HGET command
print(f"All User Data: {rclient.hgetall('user:1')}") # equivalent to Redis
HGETALL command

# Sets (Unique Collections)
print("\n--- Sets ---")
rclient.sadd('tags', 'python', 'redis', 'python') # Second 'python' element
ignored (equivalent to Redis SADD command)
rclient.sadd('other_tags', 'redis', 'database')
print(f"Tags Set:{rclient.smembers('tags')}") # equivalent to Redis SMEMBERS
command
print(f"Other Tags Set:{rclient.smembers('other_tags')}")
print(f"Common Tags: {rclient.sinter('tags', 'other_tags')}") # Intersection
(equivalent to Redis SINTER command)
print(f"Is 'coding' a tag? {rclient.sismember('tags', 'coding')}") #
equivalent to Redis SISMEMBER command)

# Sorted Sets (Leaderboards/Priorities)
print("\n--- Sorted Sets ---")
rclient.zadd('high_scores', {'player1': 100, 'player2': 500, 'player3':
250}) # equivalent to Redis ZADD command
top_players = rclient.zrange('high_scores', 0, 1, desc=True,
withscores=True) # Get top 2 players (equivalent to Redis ZRANGE command)
print(f"Leaderboard: {top_players}")

# HyperLogLogs (Massitve Unique Counts)
print("\n--- HyperLogLogs ---")
rclient.pfadd('unique_visitors', 'ip1', 'ip2', 'ip3' ,'ip1', 'ip3', 'ip1') #
equivalent to Redis PFADD command
```

```
print(f"Unique Count (Approx): {rclient.pfcount('unique_visitors')}") #
equivalent to Redis PFCOUNT command

# Pipelines (Multiple commands as batch for speed)
print("\n--- Pipelines ---")
pipe = rclient.pipeline()
pipe.set('foo', 'bar').set('baz', 'qux').incr('counter')
results = pipe.execute()
print(f"Pipeline Results: {results}")

# Manage Keys
print("\n--- Manage Keys ---")
rclient.expire('user:1', 3600) # Set expiry of 60 mins (equivalent to Redis
EXPIRE command)
print(f"Leftover time on 'user:1' key: {rclient.ttl('user:1')}")  #
equivalent to Redis TTL command
print(f"Status of 'user:1' key: {rclient.exists('user:1')}") # equivalent to
Redis EXISTS command
cursor, keys = rclient.scan(match='user:*') # Search key with matching
criteria (equivalent to Redis SCAN command)
print(f"Found keys with match 'user:*': {keys}")

# Pub/Sub (Real-time Messaging)
print("\n--- Pub/Sub ---")
pubsub = rclient.pubsub()
pubsub.subscribe('alerts') # Subscribe to a channel (equivalent to Redis
SUBSCRIBE command)
rclient.publish('alerts', 'System Rebooting!') # Publish message (equivalent
to Redis PUBLISH command)
#msg = pubsub.get_message(ignore_subscribe_messages=True) # Get actual
message recieved
msg = pubsub.get_message() # Get actual message recieved
if msg: print(f"Received Alert: {msg['data']}")
```

The above code performs the following:

- Connect to a local Redis server (`localhost:6379`) and sets `decode_responses=True` so that data comes back as Python strings instead of raw bytes.
- Perform "CRUD" operations across several Redis types such as Strings *(set values, set with expiration, increment numbers, and set multiple keys at once)*, *Lists (add items to both ends (`lpush`/`rpush`) and pops them off)*, Hashes *(store objects with multiple fields such as a user profile with `name`, `role`, and `login count`)*, *Sets (add duplicates which are ignored and find the intersection or common items between two sets)*, Sorted Sets *(create a leaderboard and retrieve the top players)*, HyperLogLogs *(estimate the count of unique items like unique website visitors)*.

- Create pipelines to batch multiple commands together to be sent to the server in one go, which significantly improves performance.
- Perform some key-management actions such as check if a key exists, set its expiration time, and search for keys matching a pattern like `user:*`.
- Implement a real-time messaging system where one part of the code "subscribes" to a channel called `alerts` and another "publishes" a message to it.




```python
import redis

# Connect to local Redis instance
rclient = redis.Redis(host='localhost', port=6379, db=0, decode_responses=True)

# Strings (Basic key-value)
print("\n--- Strings ---")
rclient.set('user:name', 'Alice') # Set a key (equivalent to Redis SET command)
rclient.set('user:email', 'alice@example.com', ex=60) # Set with 60s expiry
rclient.mset({"user:website": "example.com", "user:status": "active"}) # Multiple set (equivalent to Redis MS
rclient.incr('counter1') # Increment by 1 (equivalent to Redis INCR command)
rclient.incrby('counter1', 5) # Increment by 5 (equivalent to Redis INCRBY command)
print(f"Name: {rclient.get('user:name')}, Counter1: {rclient.get('counter1')}") # Get value (equivalent to Re

# Lists (Queues/Stacks)
print("\n--- Lists ---")
rclient.lpush('tasks', 'critical_fix','email_send', 'image_process') # Add data to front (equivalent to Redis
rclient.rpush('tasks', 'cleanup') # Add data to last (equivalent to Redis RPUSH command)
print(f"Next task: {rclient.lpop('tasks')}") # Get first record (equivalent to Redis LPOP command)
print(f"Remaining tasks: {rclient.lrange('tasks', 0, -1)}") # equivalent to Redis LRANGE command

# Hashes (Objects/Dictionaries)
print("\n--- Hashes ---")
#rclient.hset('user:1', mapping={'name': 'Alice', 'role': 'Admin', 'login_count': 5})
rclient.hset('user:1', key='name', value='Alice') # equivalent to Redis HSET command
rclient.hset('user:1', key='role', value='Admin')
rclient.hincrby('user:1', 'login_count', 1) # equivalent to Redis HINCRBY command
print(f"User1 Role: {rclient.hget('user:1', 'role')}") # equivalent to Redis HGET command
print(f"All User Data: {rclient.hgetall('user:1')}") # equivalent to Redis HGETALL command

# Sets (Unique Collections)
print("\n--- Sets ---")
rclient.sadd('tags', 'python', 'redis', 'python') # Second 'python' element ignored (equivalent to Redis SADD
rclient.sadd('other_tags', 'redis', 'database')
print(f"Tags Set:{rclient.smembers('tags')}") # equivalent to Redis SMEMBERS command
print(f"Other Tags Set:{rclient.smembers('other_tags')}")
print(f"Common Tags: {rclient.sinter('tags', 'other_tags')}") # Intersection (equivalent to Redis SINTER comm
print(f"Is 'coding' a tag? {rclient.sismember('tags', 'coding')}") # equivalent to Redis SISMEMBER command)

# Sorted Sets (Leaderboards/Priorities)
```

Go to **Command Prompt** and run the following command to execute the above script file *(replace `D:\Learning\Big Data\Redis` with your location where `redis-commands.py` file exists)*:

```
python "D:\Learning\Big Data\Redis\redis-commands.py"
```

```
C:\Windows\system32\cmd.exe
C:\Users\hp>python "D:\Learning\Big Data\Redis\redis-commands.py"

--- Strings ---
Name: Alice, Counter1: 6

--- Lists ---
Next task: image_process
Remaining tasks: ['email_send', 'critical_fix', 'cleanup']

--- Hashes ---
User1 Role: Admin
All User Data: {'name': 'Alice', 'role': 'Admin', 'login_count': '1'}

--- Sets ---
Tags Set:{'python', 'redis'}
Other Tags Set:{'redis', 'database'}
Common Tags: {'redis'}
Is 'coding' a tag? 0

--- Sorted Sets ---
Leaderboard: [('player2', 500.0), ('player3', 250.0)]

--- HyperLogLogs ---
Unique Count (Approx): 3

--- Pipelines ---
Pipeline Results: [True, True, 0]

--- Manage Keys ---
Leftover time on 'user:1' key: 3600
Status of 'user:1' key: 1
Found keys with match 'user:*': []

--- Pub/Sub ---
Received Alert: 1

C:\Users\hp>
```

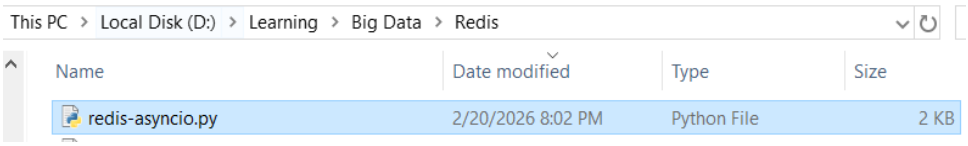
You can see the output of various data structures created in Redis database.

4.5. Async Support:

The `redis-py` library also provides support for asynchronous operations which is essential for modern, high performance and high-concurrency web frameworks like **FastAPI** or **Tornado**. It prevents the event loop from "blocking" while waiting for Redis to respond, allowing the application to handle other requests simultaneously.

The `redis.asyncio` module in `redis-py` library provides an interface almost identical to the synchronous version, but every command must be preceded by the `await` keyword and explicit call should be made using `await client.close()` to close connection pools and release resources.

Create a file named `redis-asyncio.py` in local system *(in my case, I am creating file under `D:\Learning\Big Data\Redis`)*.



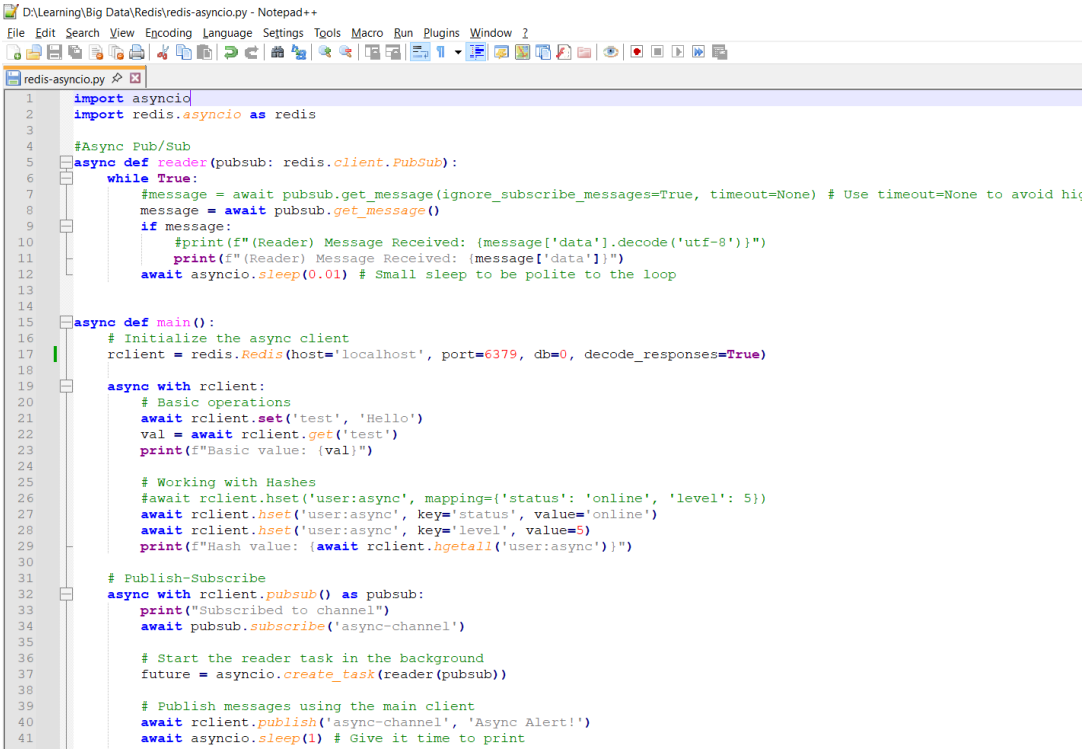
Edit `redis-asyncio.py` and add the following code:

```
import asyncio
import redis.asyncio as redis

#Async Pub/Sub
async def reader(pubsub: redis.client.PubSub):
    while True:
        #message = await pubsub.get_message(ignore_subscribe_messages=True,
timeout=None) # Use timeout=None to avoid high CPU usage
        message = await pubsub.get_message()
        if message:
            #print(f"(Reader) Message Received:
{message['data'].decode('utf-8')}")
            print(f"(Reader) Message Received: {message['data']}")
        await asyncio.sleep(0.01) # Small sleep to be polite to the loop


async def main():
    # Initialize the async client
    rclient = redis.Redis(host='localhost', port=6379, db=0,
decode_responses=True)

    async with rclient:
        # Basic operations
        await rclient.set('test', 'Hello')
        val = await rclient.get('test')
        print(f"Basic value: {val}")

        # Working with Hashes
        #await rclient.hset('user:async', mapping={'status': 'online',
'level': 5})
        await rclient.hset('user:async', key='status', value='online')
        await rclient.hset('user:async', key='level', value=5)
        print(f"Hash value: {await rclient.hgetall('user:async')}")

    # Publish-Subscribe
    async with rclient.pubsub() as pubsub:
        print("Subscribed to channel")
        await pubsub.subscribe('async-channel')

        # Start the reader task in the background
        future = asyncio.create_task(reader(pubsub))

        # Publish messages using the main client
        await rclient.publish('async-channel', 'Async Alert!')
```

```
        await asyncio.sleep(1) # Give it time to print

        await pubsub.unsubscribe("my-channel")
        future.cancel() # Stop the background task

    # Close connection
    await rclient.aclose()

# Run the event loop
if __name__ == "__main__":
    asyncio.run(main())
```

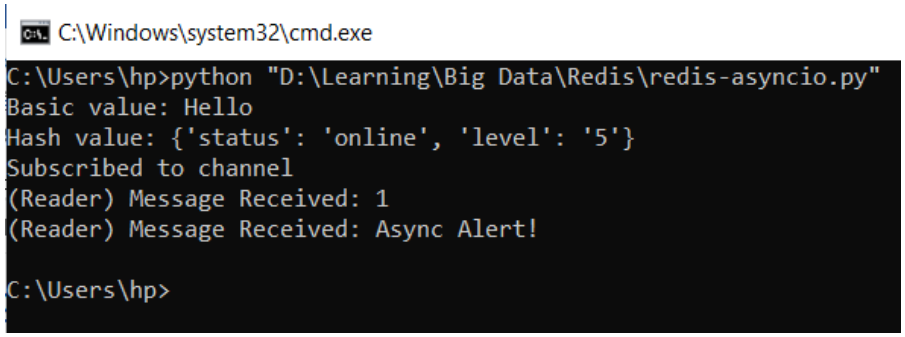
The above code performs the following:

- Connect to a local Redis server (`localhost:6379`) and set `decode_responses=True` so that data comes back as Python strings instead of raw bytes.
- Create Async context manager `async with rclient:` to ensure the connection is managed properly.
- Perform set/get operations on basic keys and hash key with `await` keyword.
- Initiate Pub/Sub by first creating an async context manager `async with rclient.pubsub()`, then starting a background reader to constantly check for new messages and sending a message to a channel (`async-channel`) so the background task catches and prints it.



Go to **Command Prompt** and run the following command to execute the above script file *(replace `D:\Learning\Big Data\Redis` with your location where `redis-aysyncio.py` file exists)*:

```
python "D:\Learning\Big Data\Redis\redis-asyncio.py"
```



You can see the output with values and messages read from channel.

5. Deploy Redis Replication:

Redis Replication deployment contains one or more secondary instances along with the main instance, following the master – slave (replica) pattern.

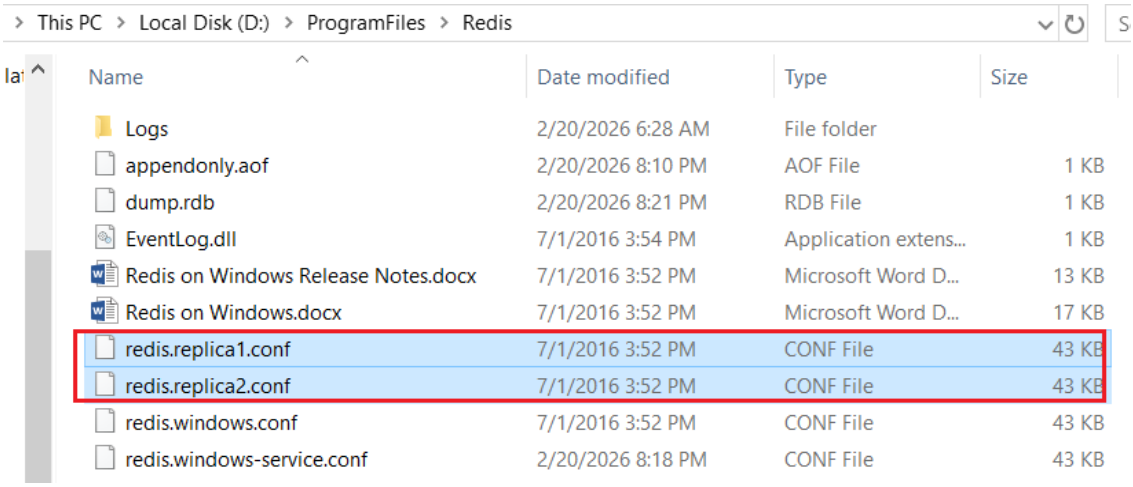
Here, we will setup Redis replication with one master and 2 replicas using the current Redis installation.

Note: The below instructions specifically apply for **Microsoft/MSOpenTech** version of Redis installed on Windows.

5.1. Prepare Configuration:

While the current Redis instance serves as a Master, another two configuration files are needed to setup two slave instances.

Go to Redis installation directory *(in my case, it is `D:\ProgramFiles\Redis`)* and take two copies of `redis.windows.conf` file and rename as `redis.replica1.conf` and `redis.replica2.conf` respectively.



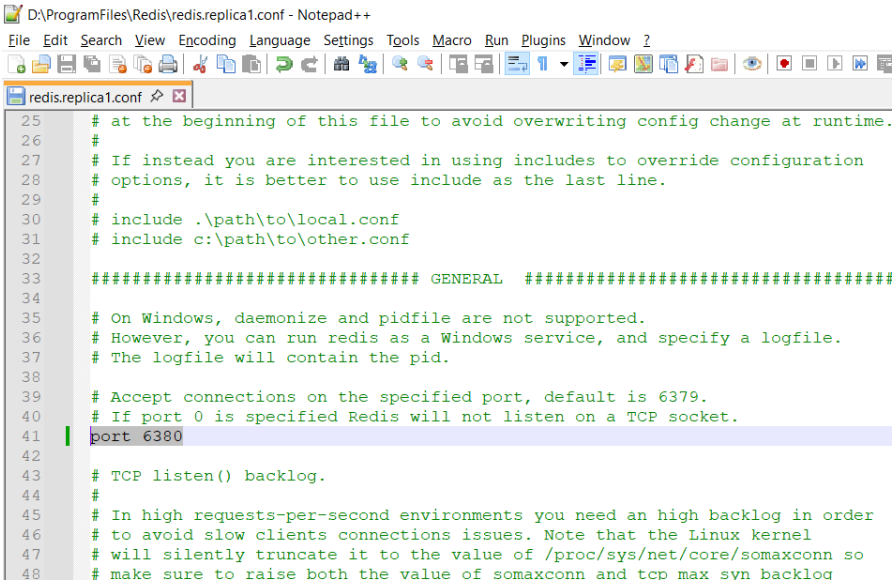
Open `redis.replica1.conf` file, and make the following changes:

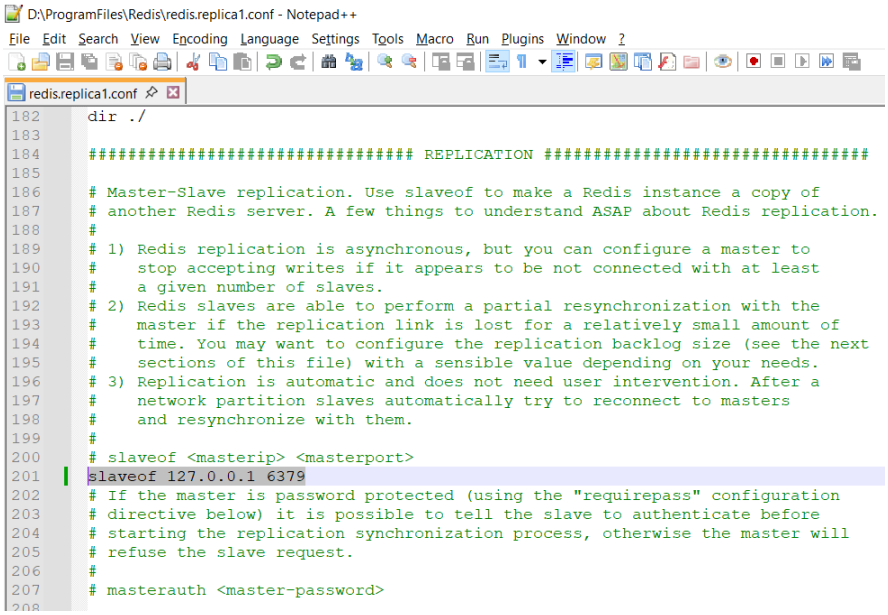
- Update port to 6380:

```
port 6380
```

- Add Replica directive as below:

```
slaveof 127.0.0.1 6379
```





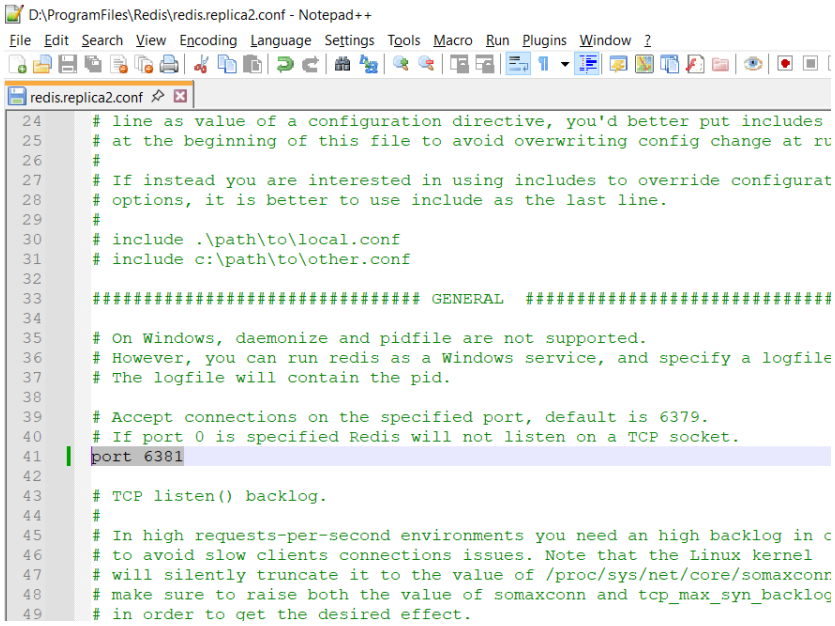
Open `redis.replica2.conf` file, and make the following changes:

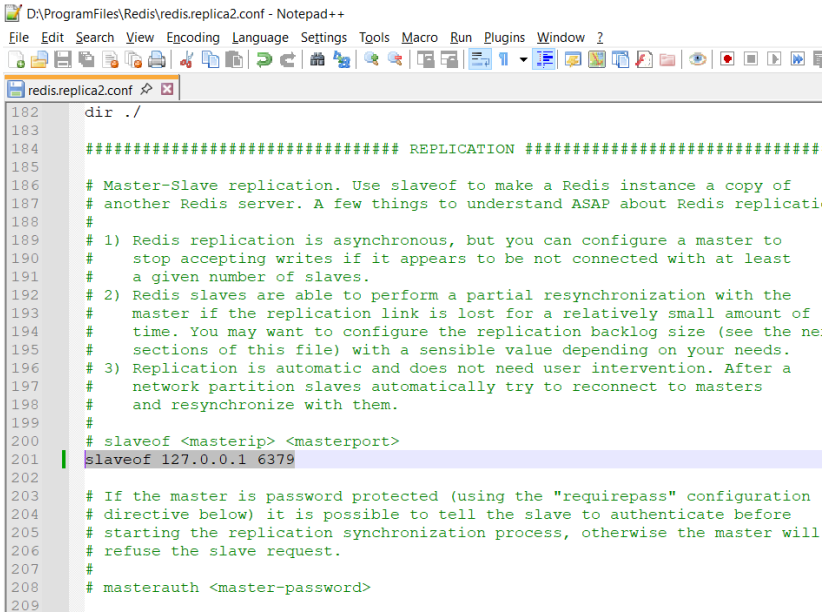
- Update port to 6381:

```
port 6381
```

- Add Replica directive as below:

```
slaveof 127.0.0.1 6379
```



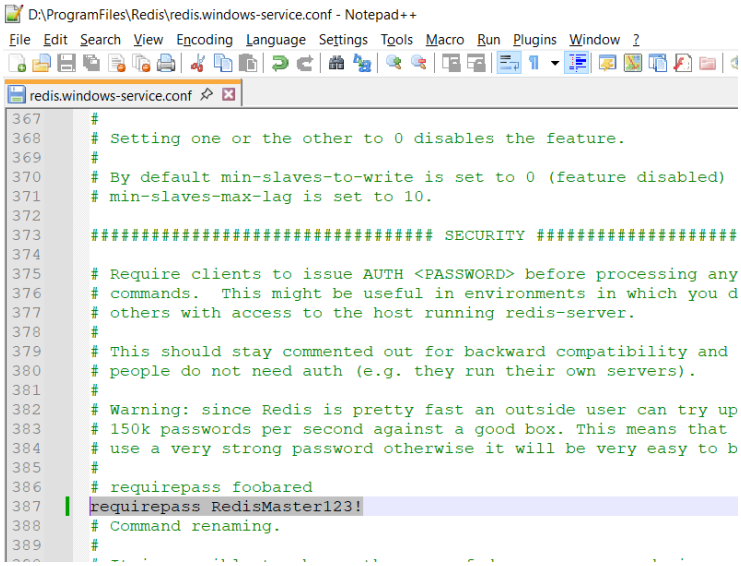


5.2. Secure Replication:

To secure the connection between Redis master and replica instances, replication credentials must be configured.

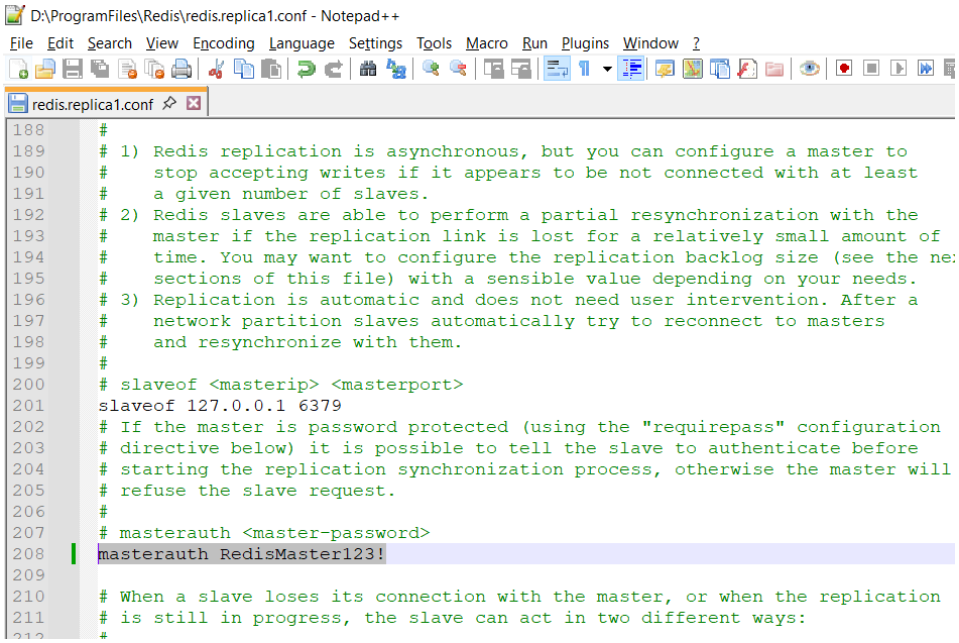
Go to Redis installation directory *(in my case, it is* `D:\ProgramFiles\Redis`*)* and edit `redis.windows-service.conf` file *(this is used by the current Redis instance that is acting as master)* and add the following to set the password for all clients and replicas that must provide to connect.

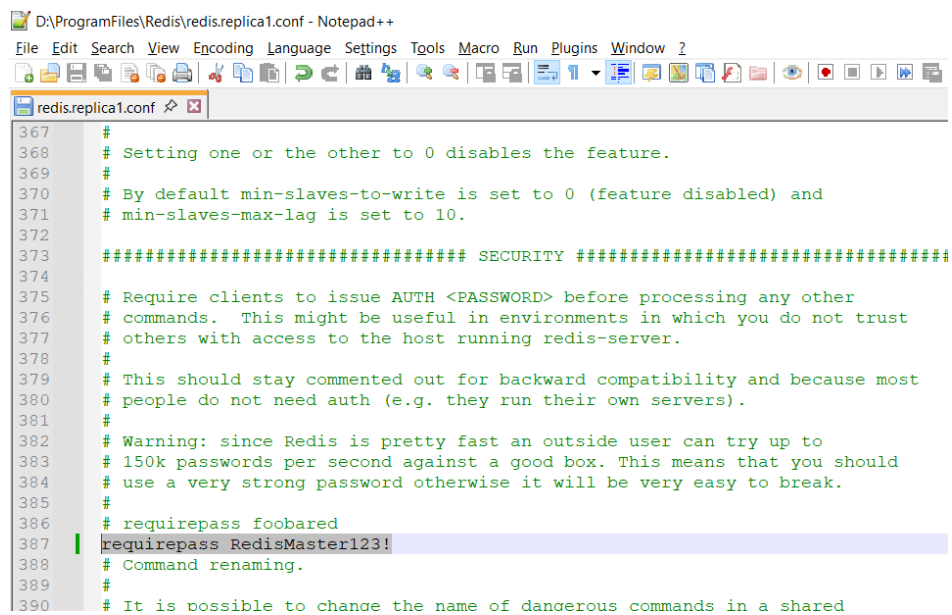
```
requirepass RedisMaster123!
```



Edit `redis.replica1.conf` file and add the following to provide master's password so it can authenticate and sync data and to set the password for this replica which is needed when this gets promoted to master:

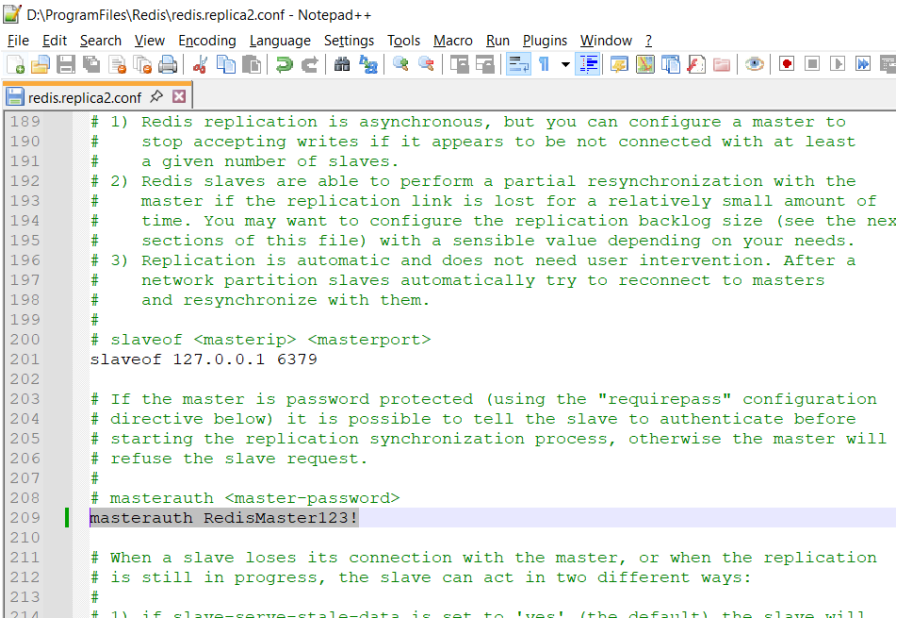
```
masterauth RedisMaster123!
requirepass RedisMaster123!
```

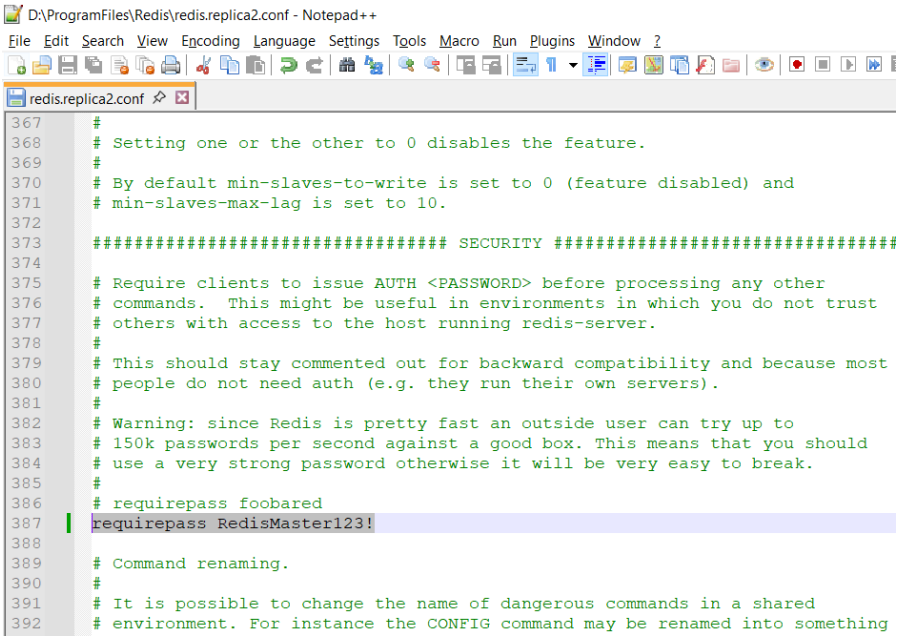




Edit `redis.replica2.conf` file and add the following to provide master's password so it can authenticate and sync data and to set the password for this replica which is needed when this gets promoted to master:

```
masterauth RedisMaster123!
requirepass RedisMaster123!
```

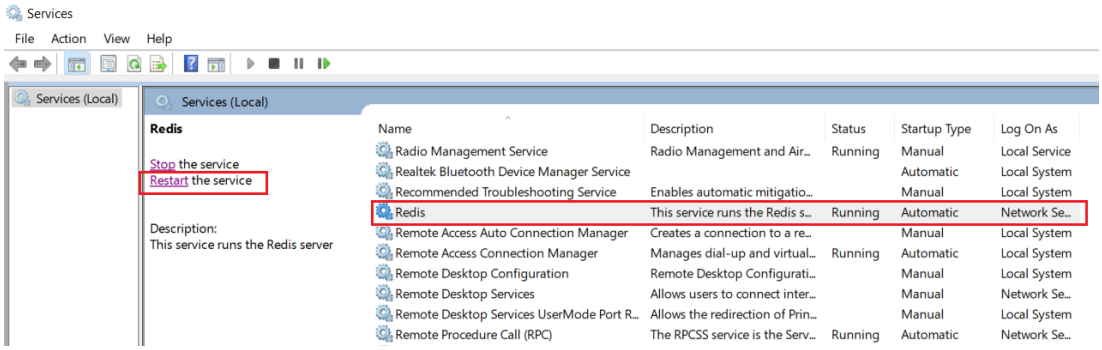




5.3. Start Instances:

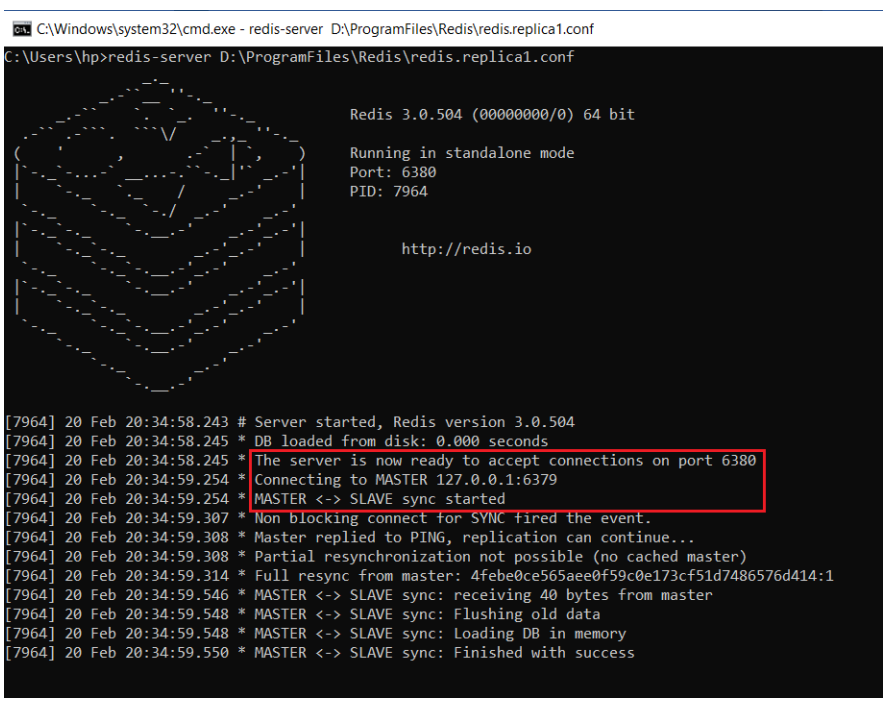
Restart master and launch replica instances.

- To restart Redis master instance, open **Services** application and click on **Restart** link at **Redis** service.



- To launch **Replica1** instance, open **Command Prompt** and run the following command *(in the below command, replace `D:\ProgramFiles\Redis` with your Redis installation path):*

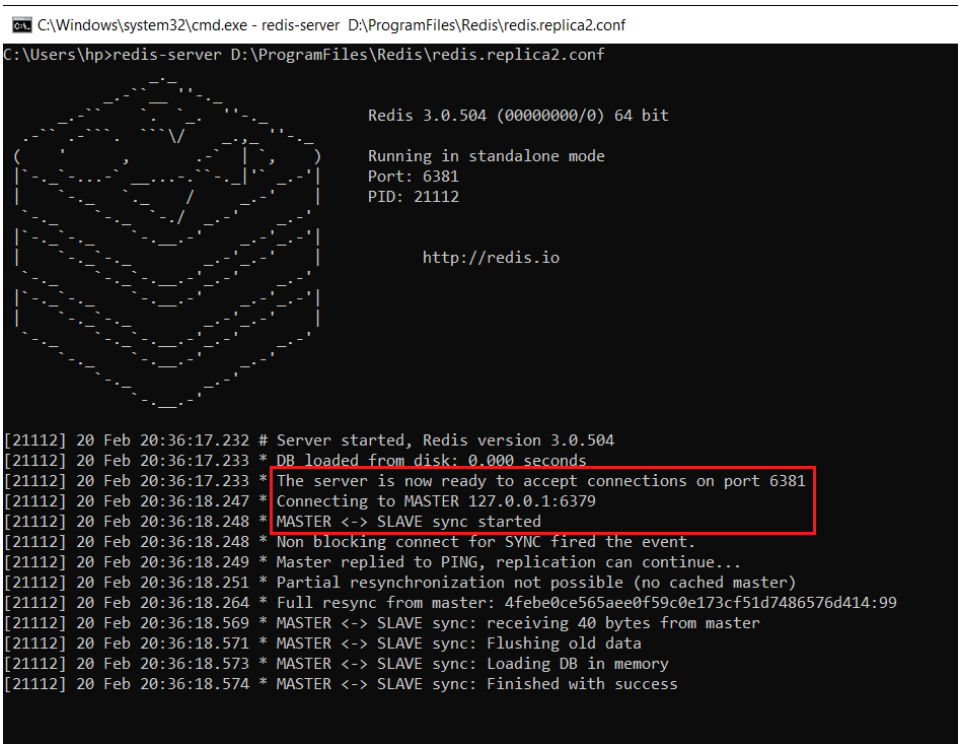
```
redis-server D:\ProgramFiles\Redis\redis.replica1.conf
```



In the log, you can see that this replica is `Connecting to MASTER 127.0.0.1:6379` as configured.

- To launch Replica2 instance, open another **Command Prompt** and run the following command *(in the below command, replace* `D:\ProgramFiles\Redis` *with your Redis installation path):*

```
redis-server D:\ProgramFiles\Redis\redis.replica2.conf
```



In the log, you can see that this replica is `Connecting to MASTER 127.0.0.1:6379` as configured.

<u>Note:</u> To start Replica instances automatically as Windows services, run the following commands:

```
redis-server --service-install redis.replica.conf --service-name
RedisReplica1
```

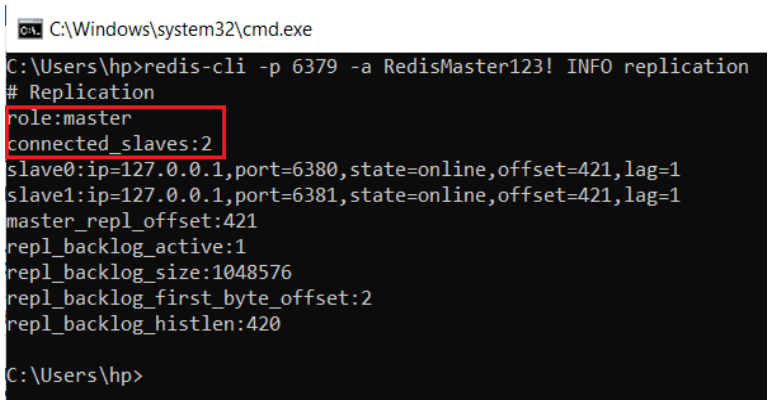
```
redis-server --service-install redis.replica.conf --service-name
RedisReplica2
```

5.4. Verify Connection:

Verify if the master and replica instances are connected and working.

Connect to the master instance and check the replication status using the following command:

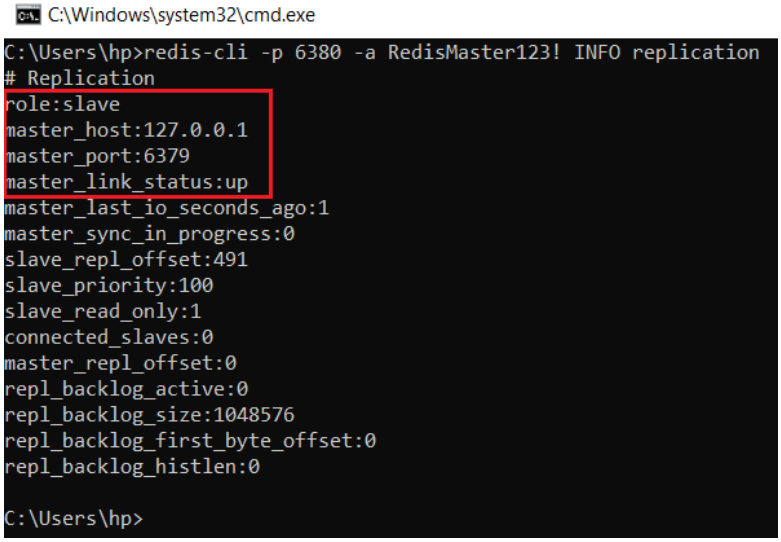
```
redis-cli -p 6379 -a RedisMaster123! INFO replication
```



It should display `role:master` and `connected_slaves:2`.

Now, connect to the Replica1 instance and check the replication status using the following command:

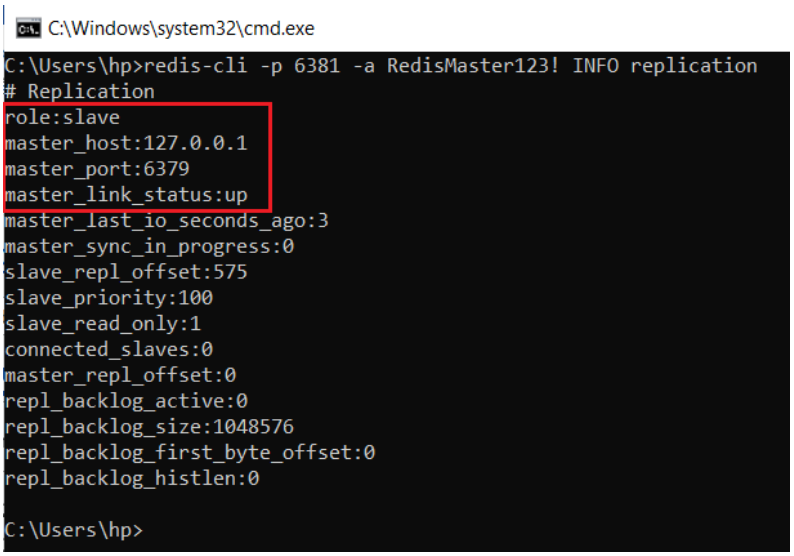
```
redis-cli -p 6380 -a RedisMaster123! INFO replication
```



It should display `role:slave` and `master_link_status:up`.

Now, connect Replica2 instance and check the replication status using the following command:

```
redis-cli -p 6381 -a RedisMaster123! INFO replication
```



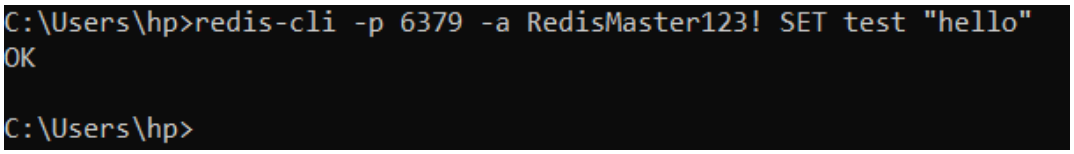
It should display `role:slave` and `master_link_status:up`.

5.5. Test Data Sync:

Verify if data sync is working as expected.

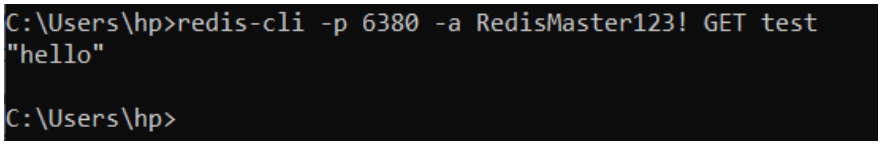
Run the following command to insert `test` key in the master instance:

```
redis-cli -p 6379 -a RedisMaster123! SET test "hello"
```



Run the following command to read `test` key from the replica1 instance:

```
redis-cli -p 6380 -a RedisMaster123! GET test
```



It should display `hello` as written to master instance.

Run the following command to read from the replica2 instance:

```
redis-cli -p 6381 -a RedisMaster123! GET test
```

```
C:\Users\hp>redis-cli -p 6381 -a RedisMaster123! GET test
"hello"

C:\Users\hp>
```

It should display `hello` as written to master instance which indicates the replication is working as expected.

6. Deploy Redis Sentinel:

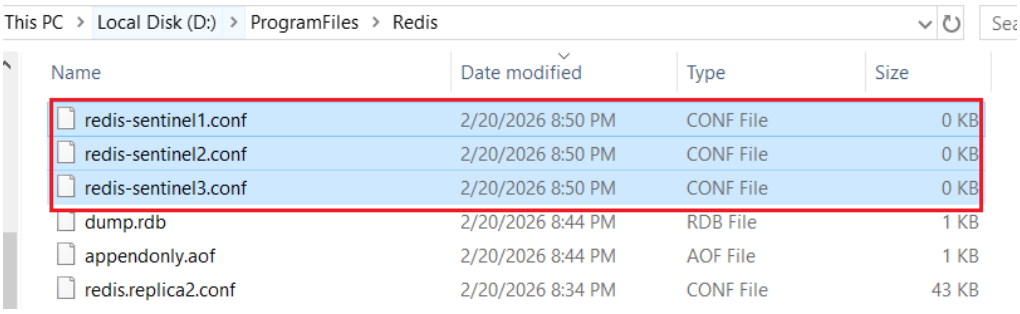
Redis Sentinel deployment requires new instances (Sentinels) to monitor Redis master and slave instances so that if master fails, Sentinels automatically promote one of the replicas to be the new master. At least three Sentinels are suggested to be configured for high availability, so if one sentinel goes down, the other two continue monitoring Redis master and replica instances and can trigger automatic failover.

For this setup, we will deploy three Sentinel instances to monitor the same master. In a production environment, this is typically achieved across three isolated servers: **Node 1** acts as the Redis Master and Sentinel 1, **Node 2** serves as Redis Replica 1 and Sentinel 2, and **Node 3** functions as Redis Replica 2 and Sentinel 3.

Note: The below instructions specifically apply for **Microsoft/MSOpenTech** version of Redis installed on Windows.

6.1. Configure Sentinel:

Go to Redis installation directory *(in my case, it is `D:\ProgramFiles\Redis`)* and create three configuration files named `redis-sentinel1.conf`, `redis-sentinel2.conf` and `redis-sentinel3.conf` respectively.



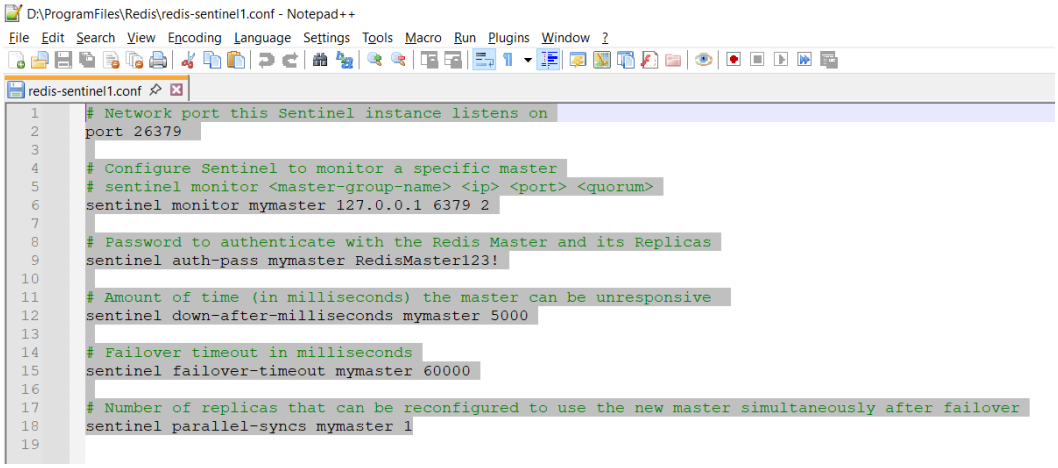
Open `redis-sentinel1.conf` file, and add the following code:

```
# Network port this Sentinel instance listens on
port 26379

# Configure Sentinel to monitor a specific master
# sentinel monitor <master-group-name> <ip> <port> <quorum>
sentinel monitor mymaster 127.0.0.1 6379 2

# Password to authenticate with the Redis Master and its Replicas
sentinel auth-pass mymaster RedisMaster123!

# Amount of time (in milliseconds) the master can be unresponsive
sentinel down-after-milliseconds mymaster 5000

# Failover timeout in milliseconds
sentinel failover-timeout mymaster 60000

# Number of replicas that can be reconfigured to use the new master
simultaneously after failover
sentinel parallel-syncs mymaster 1
```

In the above code:

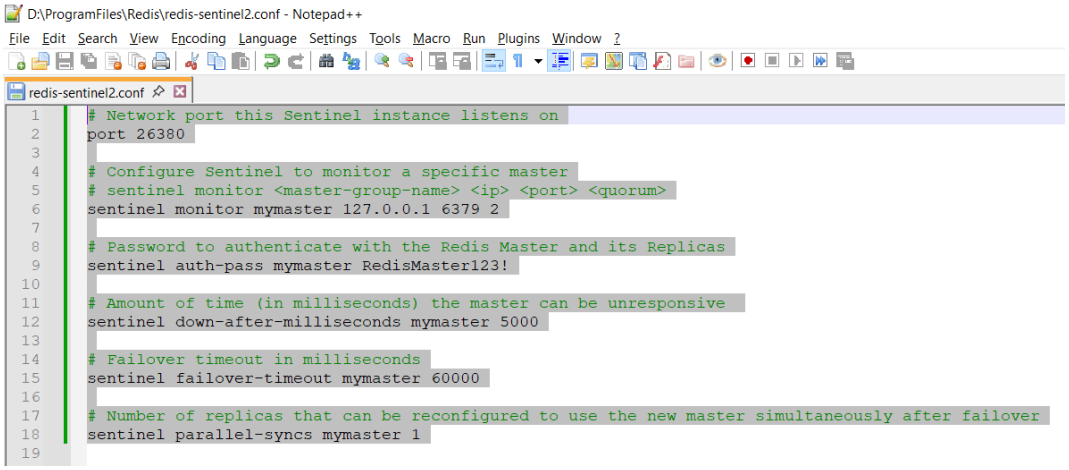
- `port 26381` indicates to run Sentinel instance on `26831` port.
- `sentinel monitor mymaster 127.0.0.1 6379 2` indicates to monitor the Redis master (named as `mymaster`) running at host `127.0.0.1` (i.e. localhost) and port `6379` with a quorum of `2` so that at least 2 sentinels agree on master failure.
- `sentinel auth-pass mymaster RedisMaster123!` Indicates to authenticate Redis master group `mymaster` with password `RedisMaster123!`.
- `sentinel down-after-milliseconds mymaster 5000` indicates to consider master down after `5` seconds of no response.
- `sentinel failover-timeout mymaster 60000` indicates to wait for `60` seconds for a failover to complete before another Sentinel can try to failover again.

- `sentinel parallel-syncs mymaster 1` indicates to sync only `1` replica at a time during the failover to avoid overwhelming the new master.



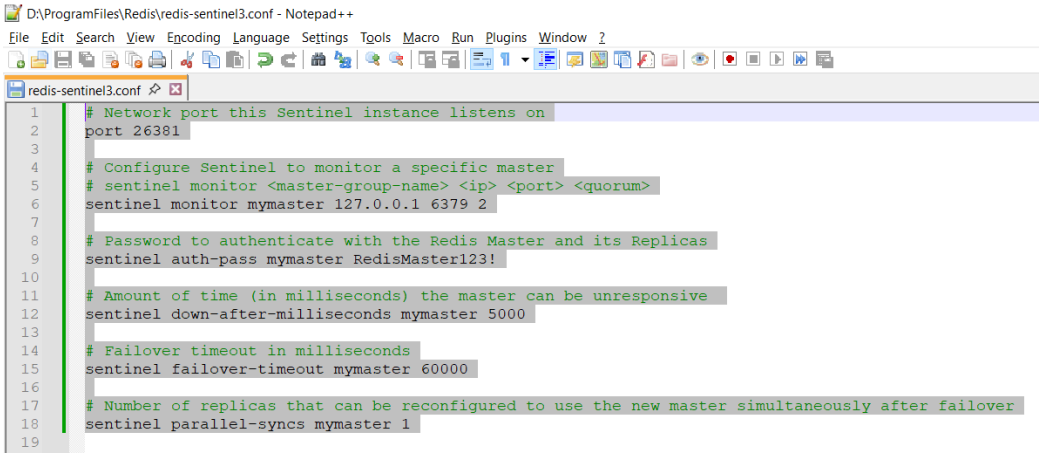
Similarly, open `redis-sentinel2.conf` file, and add the same code as in `redis-sentinel1.conf` except the change in port number as below:

```
# Network port this Sentinel instance listens on
port 26380

# Configure Sentinel to monitor a specific master
# sentinel monitor <master-group-name> <ip> <port> <quorum>
sentinel monitor mymaster 127.0.0.1 6379 2

# Password to authenticate with the Redis Master and its Replicas
sentinel auth-pass mymaster RedisMaster123!

# Amount of time (in milliseconds) the master can be unresponsive
sentinel down-after-milliseconds mymaster 5000

# Failover timeout in milliseconds
sentinel failover-timeout mymaster 60000

# Number of replicas that can be reconfigured to use the new master
simultaneously after failover
sentinel parallel-syncs mymaster 1
```



Similarly, open `redis-sentinel3.conf` file, and add the same code as in `redis-sentinel1.conf` except the change in port number as below:

```
# Network port this Sentinel instance listens on
port 26381

# Configure Sentinel to monitor a specific master
# sentinel monitor <master-group-name> <ip> <port> <quorum>
sentinel monitor mymaster 127.0.0.1 6379 2

# Password to authenticate with the Redis Master and its Replicas
sentinel auth-pass mymaster RedisMaster123!

# Amount of time (in milliseconds) the master can be unresponsive
sentinel down-after-milliseconds mymaster 5000

# Failover timeout in milliseconds
sentinel failover-timeout mymaster 60000

# Number of replicas that can be reconfigured to use the new master
simultaneously after failover
sentinel parallel-syncs mymaster 1
```

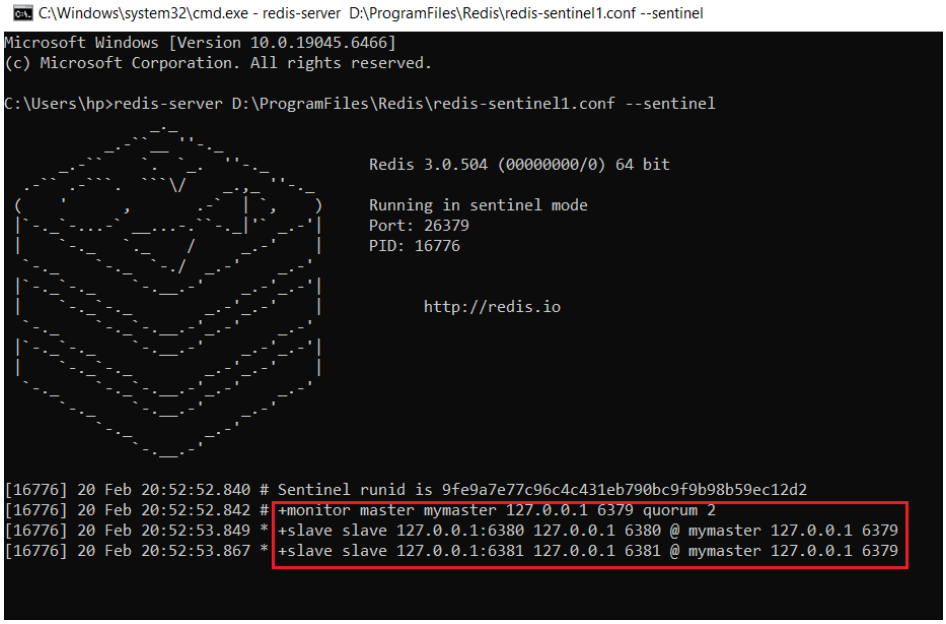


6.2. Launch Sentinel Instances:

Before starting Sentinel instances, make sure Redis master and replicas are already running. If they are not running, start Redis master first and then replica instances.

Next, open **Command Prompt** and run the following command to start Sentinel1 *(in the command below, replace `D:\ProgramFiles\Redis` with your Redis installation path):*

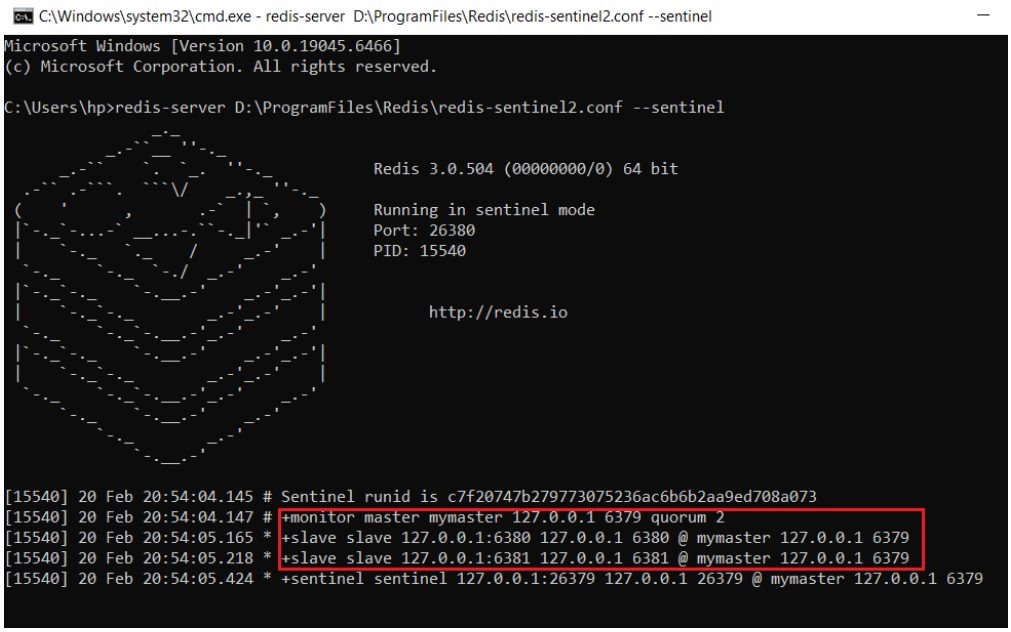
```
redis-server D:\ProgramFiles\Redis\redis-sentinel1.conf --sentinel
```



In the log, you can see that it starts monitoring the master and 2 slaves.

Open another **Command Prompt** and run the following command to start Sentinel2 *(in the command below, replace `D:\ProgramFiles\Redis` with your Redis installation path):*

```
redis-server D:\ProgramFiles\Redis\redis-sentinel2.conf --sentinel
```



In the log, you can see that it starts monitoring the master and 2 slaves.

Open another **Command Prompt** and run the following command to start Sentinel3 *(in the command below, replace `D:\ProgramFiles\Redis` with your Redis installation path):*

```
redis-server D:\ProgramFiles\Redis\redis-sentinel3.conf --sentinel
```

```
C:\Windows\system32\cmd.exe - redis-server D:\ProgramFiles\Redis\redis-sentinel3.conf --sentinel
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\hp>redis-server D:\ProgramFiles\Redis\redis-sentinel3.conf --sentinel

                Redis 3.0.504 (00000000/0) 64 bit

                Running in sentinel mode
                Port: 26381
                PID: 19416

                      http://redis.io

[19416] 20 Feb 20:54:42.112 # Sentinel runid is 9b235dfa94969e6ebab7507da97d922c47f8d346
[19416] 20 Feb 20:54:42.114 # +monitor master mymaster 127.0.0.1 6379 quorum 2
[19416] 20 Feb 20:54:43.120 * +slave slave 127.0.0.1:6380 127.0.0.1 6380 @ mymaster 127.0.0.1 6379
[19416] 20 Feb 20:54:43.162 * +slave slave 127.0.0.1:6381 127.0.0.1 6381 @ mymaster 127.0.0.1 6379
[19416] 20 Feb 20:54:43.474 * +sentinel sentinel 127.0.0.1:26379 127.0.0.1 26379 @ mymaster 127.0.0.1 6379
[19416] 20 Feb 20:54:44.071 * +sentinel sentinel 127.0.0.1:26380 127.0.0.1 26380 @ mymaster 127.0.0.1 6379
```

In the log, you can see that it starts monitoring the master and 2 slaves.

6.3. Verify Setup:

Connect to Sentinel1 and check its health using the following command:

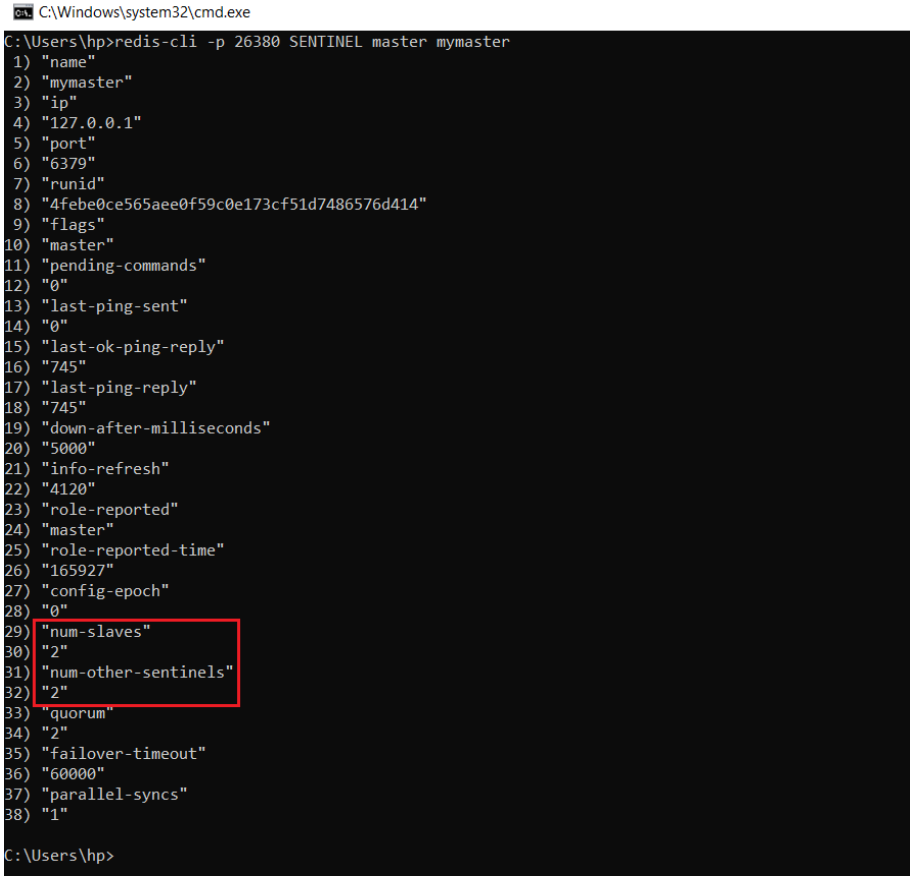
```
redis-cli -p 26379 SENTINEL master mymaster
```

```
C:\Windows\system32\cmd.exe
C:\Users\hp>redis-cli -p 26379 SENTINEL master mymaster
 1) "name"
 2) "mymaster"
 3) "ip"
 4) "127.0.0.1"
 5) "port"
 6) "6379"
 7) "runid"
 8) "4febe0ce565aee0f59c0e173cf51d7486576d414"
 9) "flags"
10) "master"
11) "pending-commands"
12) "0"
13) "last-ping-sent"
14) "0"
15) "last-ok-ping-reply"
16) "441"
17) "last-ping-reply"
18) "441"
19) "down-after-milliseconds"
20) "5000"
21) "info-refresh"
22) "919"
23) "role-reported"
24) "master"
25) "role-reported-time"
26) "152532"
27) "config-epoch"
28) "0"
29) "num-slaves"
30) "2"
31) "num-other-sentinels"
32) "2"
33) "quorum"
34) "2"
35) "failover-timeout"
36) "60000"
37) "parallel-syncs"
38) "1"

C:\Users\hp>
```

It should display `num:slaves` as `2` and `num-other-sentinels` as `2`.

Now, connect to Sentinel2 and check its health using the following command:

```
redis-cli -p 26380 SENTINEL master mymaster
```



It should display `num:slaves` as `2` and `num-other-sentinels` as `2`.

Now, connect to Sentinel3 and check its health using the following command:

```
redis-cli -p 26381 SENTINEL master mymaster
```

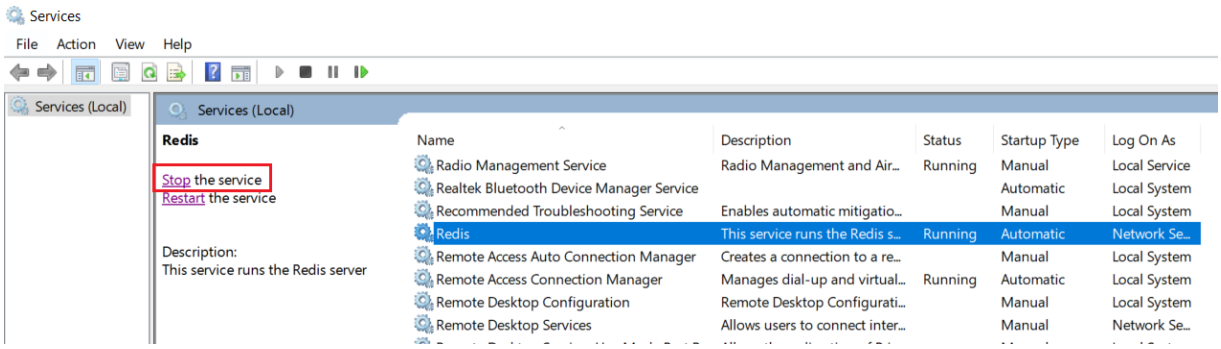
```
C:\Windows\system32\cmd.exe
C:\Users\hp>redis-cli -p 26381 SENTINEL master mymaster
 1) "name"
 2) "mymaster"
 3) "ip"
 4) "127.0.0.1"
 5) "port"
 6) "6379"
 7) "runid"
 8) "4febe0ce565aee0f59c0e173cf51d7486576d414"
 9) "flags"
10) "master"
11) "pending-commands"
12) "0"
13) "last-ping-sent"
14) "0"
15) "last-ok-ping-reply"
16) "772"
17) "last-ping-reply"
18) "772"
19) "down-after-milliseconds"
20) "5000"
21) "info-refresh"
22) "2479"
23) "role-reported"
24) "master"
25) "role-reported-time"
26) "184407"
27) "config-epoch"
28) "0"
29) "num-slaves"
30) "2"
31) "num-other-sentinels"
32) "2"
33) "quorum"
34) "2"
35) "failover-timeout"
36) "60000"
37) "parallel-syncs"
38) "1"

C:\Users\hp>
```

It should display `num:slaves` as `2` and `num-other-sentinels` as `2`.

6.4. Test Fail Over:

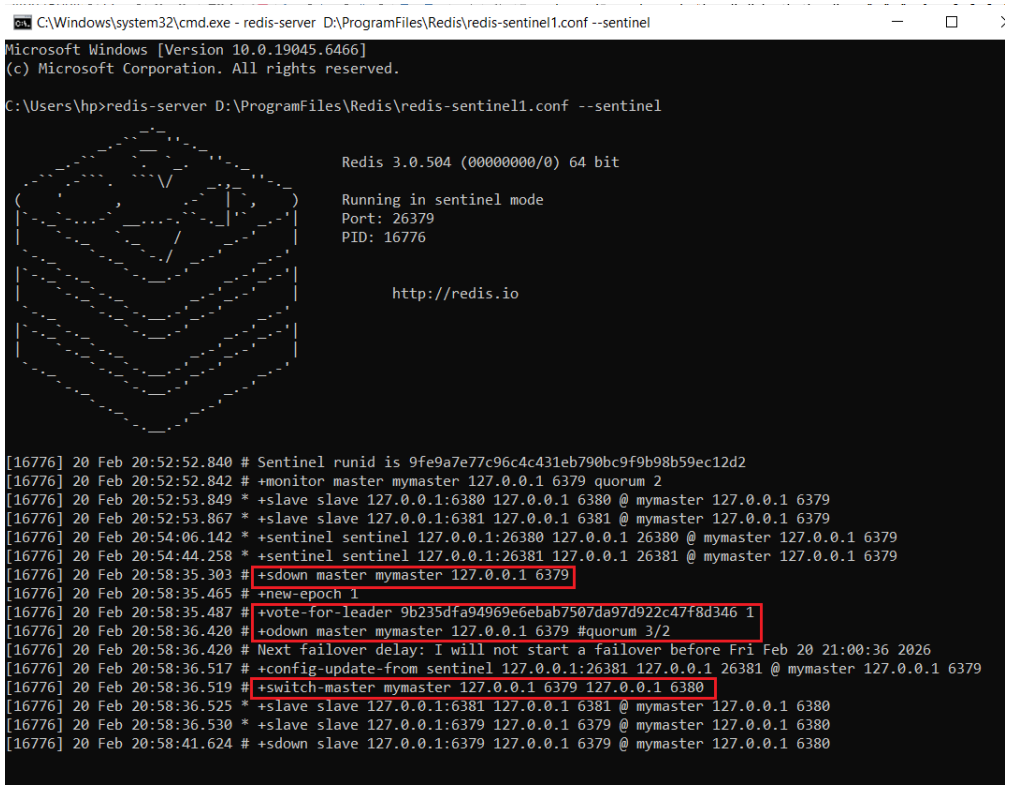
Verify if Sentinels perform the automatic failover when the Redis master is down.

Open **Services** application and click on **Stop** link at **Redis** service to stop the service.



Go to any of the Sentinel terminal windows where you can see messages such as:

- `+sdown` (Subjective Down) indicating that this Sentinel thinks the master is down.
- `+odown` (Objective Down) indicating enough Sentinels agreed based on quorum of 2.
- `+vote-for-leader` indicating the Sentinel starts the election process.
- `+switch-master` indicating the Sentinel promotes one of the replicas to be the new Master *(here, it promoted first replica running on 6380)*.



6.5. Verify New Hierarchy:

Verify the status of new master and other replica.

- **Check the promoted replica:**

  Run the following command to connect to the new master running on `6380` *(in your case, if replica running on `6381` was elected as the new master then change port to `6381` in the command below)*:

```
redis-cli -p 6380 -a RedisMaster123! INFO replication
```

```
C:\Windows\system32\cmd.exe
C:\Users\hp>redis-cli -p 6380 -a RedisMaster123! INFO replication
# Replication
role:master
connected_slaves:1
slave0:ip=127.0.0.1,port=6381,state=online,offset=26939,lag=1
master_repl_offset:27072
repl_backlog_active:1
repl_backlog_size:1048576
repl_backlog_first_byte_offset:2
repl_backlog_histlen:27071

C:\Users\hp>
```

It should display `role:master` and `connected_slaves:1`.

- **Check another replica:**

  Run the following command to connect to another replica on port `6381` *(or `6380` if your replica running on `6381` was elected as the new master)* and verify if it points to the new master:

```
redis-cli -p 6381 -a RedisMaster123! INFO replication
```

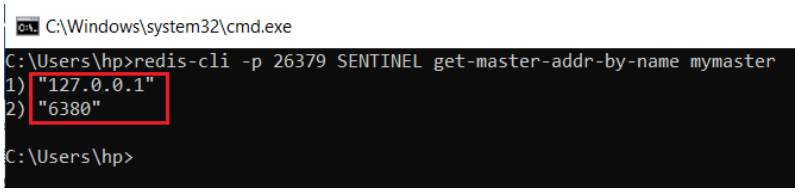
```
C:\Windows\system32\cmd.exe
C:\Users\hp>redis-cli -p 6381 -a RedisMaster123! INFO replication
# Replication
role:slave
master_host:127.0.0.1
master_port:6380
master_link_status:up
master_last_io_seconds_ago:0
master_sync_in_progress:0
slave_repl_offset:43130
slave_priority:100
slave_read_only:1
connected_slaves:0
master_repl_offset:0
repl_backlog_active:0
repl_backlog_size:1048576
repl_backlog_first_byte_offset:0
repl_backlog_histlen:0

C:\Users\hp>
```

It should display `master_host:127.0.0.1` and `master_port:6380` (*or `6381` depending on the promoted master)*.

- **Check Sentinel:**

  Run the following command to verify if Sentinel returns the new master IP and port:

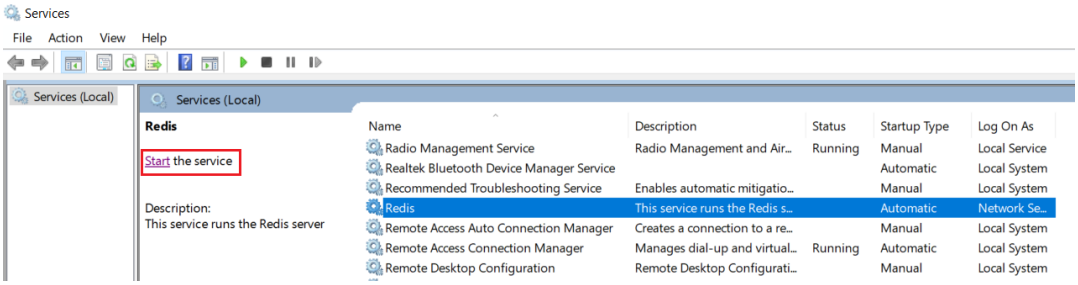
```
redis-cli -p 26379 SENTINEL get-master-addr-by-name mymaster
```



It should display `127.0.0.1` and `6380` (*or `6381` depending on the promoted master)*.

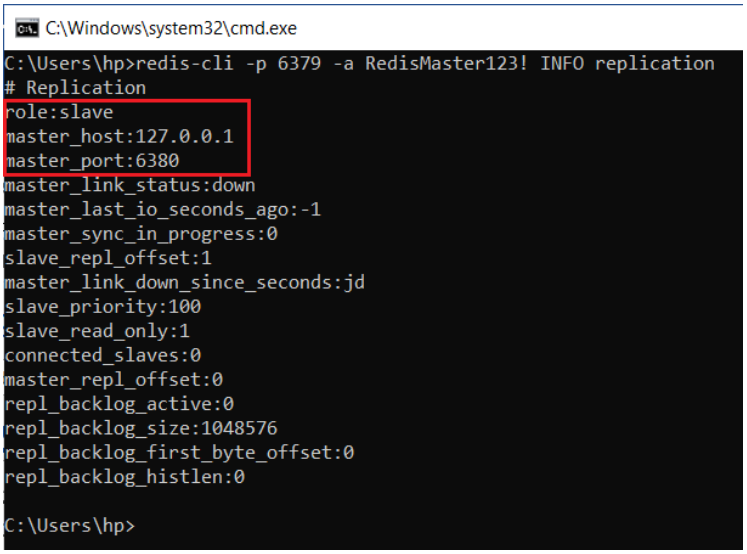
6.6. Reunion Failed Instance:

Open **Services** application and click on **Start** link at **Redis** service to start the service.



Run the following command to connect to the original master running on `6379` port:

```
redis-cli -p 6379 -a RedisMaster123! INFO replication
```



You can see that the original master role is set to `slave` because when Sentinel detects the old master back online, it does not give it the master role back and instead, it reconfigures the old master to be a replica of the new master.

6.7. Connect Sentinel from Python:

To connect to a Redis Sentinel from Python, the `Sentinel` class from `redis` library must be used. This allows the application to automatically discover the current **Master** node.

Create a file named `redis-sentinel.py` in local system *(in my case, I am creating file under `D:\Learning\Big Data\Redis`)*.

Edit `redis-sentinel.py` and add the following code:

```
from redis.sentinel import Sentinel

# Define Sentinels as list of tuples: (IP, port))
sentinel_nodes=[
    ('127.0.0.1', 26379),
    ('127.0.0.1', 26380),
    ('127.0.0.1', 26381)
    ]

# Initialize Sentinel object
sentinel=Sentinel(sentinel_nodes, socket_timeout=0.5) # `socket_timeout` is
recommended to quickly skip unreachable Sentinels

# Discover Redis Master and Replicas
master_node=sentinel.discover_master('mymaster')
replica_list=sentinel.discover_slaves('mymaster')

print(f"Master node: {master_node}")
print(f"Replicas list: {replica_list}")

# Initialize Redis master and replica client objects
master=sentinel.master_for('mymaster', socket_timeout=0.1,
password='RedisMaster123!', decode_responses=True) # Returns a standard
Redis client connected to Master
replica=sentinel.slave_for('mymaster', socket_timeout=0.1,
password='RedisMaster123!', decode_responses=True) # Returns a client
connected to one of the Replicas

# Run queries
master.set('sentinel_key', 'Hello Sentinel!')
print(f"Value from master: {master.get('sentinel_key')}")
print(f"Value from replica: {replica.get('sentinel_key')}")
```

The above code performs the following:

- Define a list of Sentinel nodes running on different ports like `26379`, `26380` and `26381`.

- Ask Sentinels to identify the current Master and its Replicas for a specific group named `mymaster`.
- Initialize the master client specifically for writing data.
- Initialize the replica client for reading data which is useful for reading data.
- Write a key to the master and then read it back from both the master and a replica to verify the connection.

D:\Learning\Big Data\Redis\redis-sentinel.py - Notepad++

File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?

redis-sentinel.py

```python
from redis.sentinel import Sentinel

# Define Sentinels as list of tuples: (IP, port))
sentinel_nodes=[
    ('127.0.0.1', 26379),
    ('127.0.0.1', 26380),
    ('127.0.0.1', 26381)
    ]

# Initialize Sentinel object
sentinel=Sentinel(sentinel_nodes, socket_timeout=0.5) # `socket_timeout` is recommended to quickly skip unreachable Sentinel

# Discover Redis Master and Replicas
master_node=sentinel.discover_master('mymaster')
replica_list=sentinel.discover_slaves('mymaster')

print(f"Master node: {master_node}")
print(f"Replicas list: {replica_list}")

# Initialize Redis master and replica client objects
master=sentinel.master_for('mymaster', socket_timeout=0.1, password='RedisMaster123!!', decode_responses=True) # Returns a st
replica=sentinel.slave_for('mymaster', socket_timeout=0.1, password='RedisMaster123!!', decode_responses=True) # Returns a cl

# Run queries
master.set('sentinel_key', 'Hello Sentinel!')
print(f"Value from master: {master.get('sentinel_key')}")
print(f"Value from replica: {replica.get('sentinel_key')}")
```

Open **Command Prompt** and run the following command to execute the above script file *(replace `D:\Learning\Big Data\Redis` with your location where `redis-sentinel.py` file exists)*:

```
python "D:\Learning\Big Data\Redis\redis-sentinel.py"
```

C:\Windows\system32\cmd.exe

```
C:\Users\hp>python "D:\Learning\Big Data\Redis\redis-sentinel.py"
Master node: ('127.0.0.1', 6380)
Replicas list: [('127.0.0.1', 6379), ('127.0.0.1', 6381)]
Value from master: Hello Sentinel!
Value from replica: Hello Sentinel!

C:\Users\hp>
```

Congratulations!! You have successfully installed Redis server along with configuration of Replication and Sentinel deployments and accessed Redis database from Python in Windows operating system.